包含“王”的记录。

（9）in

```
in(R column, Collection<?> value)
```

in()方法用于匹配字段的值在指定组合中的记录，例如，in("age",{1,2,3})用于匹配 age 字段中值为 1 或者 2 或者 3 的记录。

（10）groupBy

```
groupBy(R... columns)
```

groupBy()方法用于给指定字段进行分组，例如，groupBy("id", "name")用于对记录根据 id 字段和 name 字段进行分组。

条件构造器的方法比较多，上述方法为条件构造器常用的方法，如果读者有需要使用条件构造器更多的方法，可以参考 MyBatis-Plus 官网中的文档。

5.3.3 整合 MyBatis-Plus

MyBatis-Plus 能够显著提高程序员的开发效率，在实现数据访问时代码更简洁。在编写 MyBatis-Plus 代码时，需要注重数据的安全性和规范性，建立完整、严格的审计机制，确保程序正常运行并避免异常流动。同时，应注重科学精神，不断学习、不断研究新的技术、新的规范，使程序更加优秀，更符合现代化的需求。

下面将 Spring Boot 和 MyBatis-Plus 进行整合，并进一步演示 MyBatis-Plus 在 Spring Boot 项目中的基本使用，具体如下。

1. 配置依赖

在项目 chapter05 的 pom.xml 文件中配置 MyBatis-Plus 整合 Spring Boot 的启动器依赖，由于该依赖不是 Spring Boot 提供的，需要自行配置对应的依赖版本号，具体如文件 5-8 所示。

文件 5-8 pom.xml

```
<dependency>
    <groupId>com.baomidou</groupId>
    <artifactId>mybatis-plus-boot-starter</artifactId>
    <version>3.4.0</version>
</dependency>
```

2. 设置配置信息

MyBatis-Plus 整合 Spring Boot 的启动器依赖，提供了数据库连接的一些默认配置。如果读者有单独的 MyBatis 配置，请将对应的配置文件路径配置到 configLocation 中，例如以下配置。

```
mybatis-plus:
  config-location: classpath:mybatis-config.xml
```

在上述配置中，指定加载 classpath 下的 MyBatis 配置文件 mybatis-config.xml，本案例只演示 MyBatis-Plus 的基本操作，不需要配置额外的 MyBatis 配置，所以直接使用文件 5-2 中原有的数据库连接配置即可。

3. 创建实体类

MyBatis-Plus 标注实体类的注解与 Spring Data JPA 不一样，所以需要使用 MyBatis-Plus 的注解标注实体类。在 com.itheima.chapter05.entity 包下创建实体类 EBook，具体如文件 5-9 所示。

文件 5-9　EBook.java

```
1  import com.baomidou.mybatisplus.annotation.TableId;
2  import com.baomidou.mybatisplus.annotation.TableName;
3  @TableName("book")
4  public class EBook {
5      @TableId(type = IdType.AUTO)
6      private Integer id;        //图书编号
7      private String name;       //图书名称
8      private String author;     //图书作者
9      private String press;      //图书出版社
10     private String status;     //图书状态
11      //setter/getter 方法，以及 toString()方法
12 }
```

在上述代码中，第 3 行代码使用@TableName 注解指定当前实体类对应的数据表名称，建立实体类和数据表的映射关系；第 5 行代码使用@TableId 注解声明当前属性对应数据表的主键，并指定主键策略为自动增长。

MyBatis-Plus 默认开启驼峰命名映射，数据表和实体类进行映射时，如果数据表名包含下画线，会去掉数据表名的下画线，并将下画线后的单词首字母变为大写字母后，再匹配对应的实体类。如果实体类的字段名与数据库中的字段名不匹配，可以在属性的上方使用@TableField 注解指定对应的字段名。如果属性名称和字段名称一致，可以省略。

4. 自定义 Mapper 接口

在 com.itheima.chapter05.dao 包下创建自定义接口 BookMapper，并使用该接口继承 BaseMapper 接口，具体如文件 5-10 所示。

文件 5-10　BookMapper.java

```
1  import com.baomidou.mybatisplus.core.mapper.BaseMapper;
2  import com.itheima.chapter05.entity.EBook;
3  public interface BookMapper extends BaseMapper<EBook> {
4  }
```

在上述代码中，第 3 行代码指定操作的实体类为 EBook。

如果 Mapper 中有自定义方法，并提供了对应 XML 映射文件加以实现，就需要配置 Mapper 所对应的 XML 文件位置。例如以下配置。

```
  mybatis-plus:
 mapper-locations: classpath*:mybatis/*.xml
```

在上述配置中，指定加载 classpath 的 mybatis 目录下所有的映射文件，本案例没有自定义其他方法，故无须进行上述配置。

5. 创建 Service 接口和实现类

在项目的 java 目录下创建包 com.itheima.chapter05.service ,并在该包下创建 Service 接口和实现类，具体如文件 5-11 和文件 5-12 所示。

文件 5-11　BookService.java

```
1  import com.baomidou.mybatisplus.extension.service.IService;
2  import com.itheima.chapter05.entity.EBook;
3  public interface BookService  extends IService<EBook> {
4  }
```

在上述代码中，第 3 行代码表示自定义的 BookService 接口继承了 IService 接口，并指

定操作的实体类为 EBook。

文件 5-12 BookServiceImpl.java

```
1 import com.baomidou.mybatisplus.extension.service.IService;
2 import com.baomidou.mybatisplus.extension.service.impl.ServiceImpl;
3 import com.itheima.chapter05.dao.BookMapper;
4 import com.itheima.chapter05.entity.EBook;
5 public class BookServiceImpl extends ServiceImpl<BookMapper, EBook>
6         implements BookService {
7 }
```

在上述代码中，第 5～6 行代码表示自定义的 BookServiceImpl 类继承了 ServiceImpl 类，并且实现了 BookService，指定注入的 BaseMapper 实例对象为 BookMapper，操作的实例对象的类型为 EBook。

6. 扫描 Mapper 接口

在启动类 Chapter05Application 上方使用@MapperScan 注解扫描指定路径的 Mapper 并交由 Spring 管理，具体如文件 5-13 所示。

文件 5-13 Chapter05Application.java

```
1 import org.mybatis.spring.annotation.MapperScan;
2 import org.springframework.boot.SpringApplication;
3 import org.springframework.boot.autoconfigure.SpringBootApplication;
4 @MapperScan("com.itheima.chapter05.dao")
5 @SpringBootApplication
6 public class Chapter05Application {
7     public static void main(String[] args) {
8         SpringApplication.run(Chapter05Application.class, args);
9     }
10 }
```

在上述代码中，第 4 行代码用于指定扫描 com.itheima.chapter05.dao 包下的 Mapper，运行时会为该包下面的所有 Mapper 生成相应的实现类。

7. 定义测试方法

在 src\test\java 目录的 com.itheima.chapter05 包下创建测试类 Chapter05ApplicationMPTests，在 Chapter05ApplicationMPTests 测试类中定义操作图书信息的测试方法，具体如文件 5-14 所示。

文件 5-14 Chapter05ApplicationMPTests.java

```
1 import com.baomidou.mybatisplus.core.conditions.query.QueryWrapper;
2 import com.itheima.chapter05.dao.BookMapper;
3 import com.itheima.chapter05.entity.EBook;
4 import com.itheima.chapter05.service.BookService;
5 import org.junit.jupiter.api.Test;
6 import org.springframework.beans.factory.annotation.Autowired;
7 import org.springframework.boot.test.context.SpringBootTest;
8 import java.util.List;
9 @SpringBootTest
10 class Chapter05ApplicationMPTests {
11     @Autowired
12     private BookMapper bookMapper;
13     @Autowired
```

```
14    private BookService bookService;
15    private  void booksInfo(){
16        List<EBook> ebooks = bookMapper.selectList(null);
17        for (EBook ebook : ebooks) {
18            System.out.println(ebook);
19        }
20    }
21    @Test
22    void saveEBook() {
23        EBook ebook=new EBook(null,"人间词话","王国维","四川文艺出版社","1");
24        //新增图书信息
25        bookMapper.insert(ebook);
26        booksInfo();
27    }
28    @Test
29    void findEBook() {
30        QueryWrapper<EBook> wrapper = new QueryWrapper<>();
31        wrapper.eq("status",1).like("name","词");
32        //根据图书状态和图书名称查找对应的图书
33        List<EBook> eBooks = bookMapper.selectList(wrapper);
34        for (EBook ebook : eBooks) {
35            System.out.println(ebook);
36        }
37    }
38    @Test
39    void editEBook() {
40        System.out.println("--------图书修改前--------");
41        booksInfo();
42        EBook ebook = bookService.getById(4);
43        ebook.setName("楚辞");
44        ebook.setPress("中华书局");
45        //根据图书编号修改图书
46        bookService.updateById(ebook);
47        System.out.println("--------图书修改后--------");
48        booksInfo();
49    }
50    @Test
51    void delEBook() {
52        //根据图书编号删除图书信息
53        bookService.removeById(4);
54        booksInfo();
55    }
56 }
```

在上述代码中，第 11～14 行代码注入了 BookMapper 对象和 BookService 对象；第 15～55 行代码定义了 5 个方法。其中，第 15～20 行代码的 booksInfo()方法中调用 BookMapper 的 selectList()方法查询所有图书信息，并输出到控制台。

第 22～27 行代码的 saveEBook()方法用于测试新增图书信息，新增图书后调用 booksInfo()方法查询所有图书信息。

第 29～37 行代码的 findEBook()方法用于测试查询图书信息，第 30 行代码用于创建一

个条件构造器，第 31 行代码用于设置条件构造器的条件，第 33 行代码用于根据条件构造器中设置的条件查询图书信息。

第 39～49 行代码的 editEBook()方法用于测试修改图书信息，第 42 行代码调用 BookService 的 getById()方法获取对应的图书信息，第 46 行代码将修改后的图书信息传入 updateById()方法用于修改图书信息。

第 51～55 行代码的 delEBook()方法中，调用 BookService 的 removeById()方法，根据图书编号删除对应的图书信息。

文件 5-14 中 BookMapper 和 BookService 调用的所有方法都是直接继承自通用 Mapper 和通用 Service，并未手动编写和实现任何自定义的方法。

8. 测试操作图书信息

运行文件 5-14 中的 saveEBook()方法，测试新增图书信息，控制台输出如图 5-11 所示。

图5-11 新增图书信息（2）

从图 5-11 可以看出，控制台输出了 4 条图书信息，其中第 4 条信息为新增的图书信息，说明执行 saveEBook()方法后成功在数据库中新增了图书信息。

运行文件 5-14 中的 findEBook()方法，测试查询图书信息，控制台输出如图 5-12 所示。

图5-12 查询图书信息（2）

从图 5-12 可以看出，控制台输出了两条图书信息，图书状态为 1，图书名称都包含关键字“词”，说明执行 findEBook()方法后成功根据条件构造器查询出对应的图书信息。

运行文件 5-14 中的 editEBook()方法，测试修改图书信息，控制台输出如图 5-13 所示。

图5-13 修改图书信息（2）

从图 5-13 可以看出，图书编号为 4 的图书信息已被成功修改了图书名称和图书出版社。

运行文件 5-14 中的 delEBook()方法，测试删除图书信息，控制台输出如图 5-14 所示。

从图 5-14 可以看出，控制台输出了图书编号为 2、3、5 的 3 条图书信息，说明执行

delEBook()方法后成功删除了图书编号为 4 的图书信息。

图5-14　删除图书信息（2）

至此，Spring Boot 整合 MyBatis-Plus 后，在项目中演示 MyBatis-Plus 的基本使用已经完成。

5.4　Spring Boot 整合 Redis

随着互联网 Web 2.0 的兴起，关系型数据库在处理超大规模和高并发的 Web 2.0 网站的数据时存在一些不足，需要采用更适合解决大规模数据集合、多重数据种类的数据库，通常将这种类型的数据库统称为非关系型数据库（Not Only SQL，NoSQL）。常见的非关系型数据库有 MongoDB、Redis 等，其中 Redis 以超高的性能、完美的文档和简洁易懂的源码受到广大开发人员的喜爱。下面将对 Redis 的相关知识和 Spring Boot 整合 Redis 进行讲解。

5.4.1　Redis 快速入门

为了读者能对 Redis 快速入门，下面对 Redis 概述、Redis 安装和启动、Redis 的数据类型分别进行讲解。

1. Redis 概述

Redis（Remote Dictionary Server，远程字典服务）是一个基于内存的键值型非关系型数据库，以 Key-Value 的形式存储数据。Redis 中存储键（Key）、值（Value）的方式与 Java 中的 HashMap 类似，键和值是映射关系。在同一个库中，Key 是唯一不可重复的，每一个 Key 对应一个 Value。键值存储的本质就是使用 Key 标示 Value，当想要检索 Value 时，必须使用与 Value 相对应的 Key 进行查找。

Redis 与传统的关系型数据库截然不同，Redis 没有提供手动创建数据库的语句，Redis 启动后会默认创建 16 个数据库，用 0～15 进行编号，默认使用编号为 0 的数据库。

相较于其他的键值存储系统，Redis 主要有以下优点。

- 存取速度快：Redis 基于内存来实现数据存取，相对于磁盘来说，其读写速度要高出好几个数量级，每秒可执行大约 110000 次的设置操作，或者执行 81000 次的读取操作。
- 支持丰富的数据类型：Redis 支持开发人员常用的大多数数据类型，例如列表、集合、有序集合和散列等。
- 操作具有原子性：所有 Redis 操作都是原子操作，这使得当两个客户端并发访问时，Redis 服务器能接收更新后的值。
- 提供多种功能：Redis 提供了多种功能特性，可用作非关系型数据库、缓存中间件、消息中间件等。

2. Redis 安装和启动

要想使用非关系型数据库 Redis，必须先安装 Redis。Redis 可以在 Windows 系统和 Linux

系统安装，也可以通过 Docker 镜像来安装，不同安装方式的安装过程也不相同。为了方便操作，此处选择在 Windows 平台下进行 Redis 安装。

Redis 官方网站没有提供 Windows 版的安装包，但微软开源技术团队在 GitHub 中提供了 Windows 版的安装包。本书对应的资源中提供了从 GitHub 中下载的 ZIP 格式的 Windows 版安装包，读者可以直接使用资源中的安装包或者自行到 GitHub 下载安装包。

将 Redis 安装包解压到自定义目录下即可，不需要进行额外配置。其对应目录文件内容如图 5-15 所示。

图5-15 Redis安装包目录文件内容

从图 5-15 可以看出，Redis 安装包解压后有多个文件，这些文件中 redis-server.exe 用于启动 Redis 服务，redis-cli.exe 是 Redis 提供的客户端工具，用于启动 Redis 客户端程序。

双击 redis-server.exe，启动 Redis 服务，效果如图 5-16 所示。

从图 5-16 可以看出，Redis 服务正常启动，同时在终端窗口显示了当前 Redis 版本为 5.0.14.1 和默认启动端口号为 6379。

启动 Redis 服务后，双击 redis-cli.exe 启动客户端程序，效果如图 5-17 所示。

从图 5-17 可以看出，命令行窗口中显示 IP 地址并且光标闪烁，说明 Redis 本地客户端与服务器端连接成功。

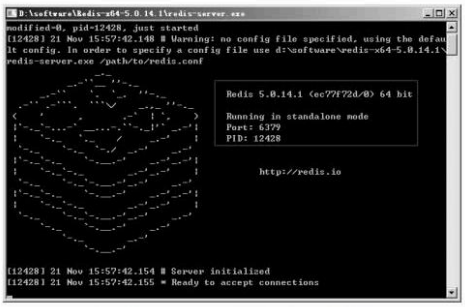

图5-16 Redis服务启动效果

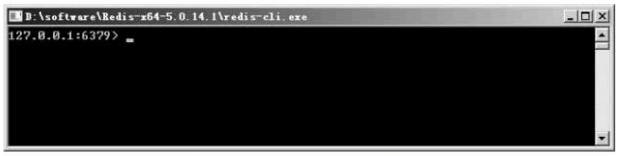

图5-17 启动客户端程序

Redis 自带的客户端工具有时候使用起来并不是特别方便，读者也可以使用一些图形化 Redis 客户端管理软件管理 Redis。常用的有 Redis Desktop Manager，其在 2022 年更名为

RESP.app，本书对应的资源中提供了该软件的安装包，安装过程很简单，读者可以自行安装。安装好后启动 RESP.app，主界面如图 5-18 所示。

在图 5-18 中，单击左侧的“连接到 Redis 服务器”，弹出“新连接设置”对话框，如图 5-19 所示。

图5-18　RESP.app主界面

图5-19　“新连接设置”对话框

在图 5-19 中，连接名可以自定义，在此设置为“SpringBoot”，设置所连接的 Redis 的相关信息后，单击“确定”按钮创建连接。

从图 5-20 可以看出，成功创建了一个名称为 SpringBoot 的连接，并且自动创建了名称为 db0～db15 的数据库。

图5-20　新建连接

3. Redis 的数据类型

Redis 中没有“数据表”的概念，通过 Value 不同的数据类型来实现存储数据的需求，不同的数据类型能够适应不同的应用场景，从而满足开发者的需求。Value 的数据类型有五种，分别为 String（字符串）、Hash（散列）、List（列表）、Set（集合）、SortedSet（有序集合），下面对这些数据类型分别进行讲解。

（1）String

String 可以灵活地表示字符串、整数、浮点数这 3 种值，String 有以下常见命令。

- SET key value：添加或者修改已经存在的键值对。
- GET key：根据键获取对应的值。
- MSET key1 value1 [key2 value2 ...]：批量添加多个键值对。
- MGET key1 [key2..]：根据一个或多个键获取对应的值。

- INCR key：将键存储的整数值自增 1。
- INCRBY key increment：将键存储的整数值根据指定步长 increment 自增。
- INCRBYFLOAT key increment：将键存储的浮点数根据指定步长 increment 自增。
- SETNX key value：当且仅当 Key 不存在时，添加一个键值对。
- SETEX key seconds value：添加一个 String 类型的键值对，并且指定有效期为 seconds 秒。

（2）List

Redis 中的 List 与 Java 中的 LinkedList 类似，可以看作是一个双向链表结构，既可以支持正向检索，也可以支持反向检索。List 类型的数据有序、元素可以重复、插入和删除速度快、查询速度一般，常见命令如下。

- LPUSH key value1 [value2...]：根据键向列表左侧插入一个或多个元素。
- LPOP key：根据键移除并返回列表左侧的第一个元素，没有则返回 nil。
- RPUSH key value1 [value2 ...]：根据键向列表右侧插入一个或多个元素。
- RPOP key：根据键移除并返回列表右侧的第一个元素。
- LRANGE key star end：根据键返回指定范围内的所有元素。
- BLPOP 和 BRPOP：与 LPOP 和 RPOP 类似，只不过在没有元素时等待指定时间，而不是直接返回 nil。

（3）Set

Redis 的 Set 类型与 Java 中的 HashSet 类似，可以看作是一个 Value 为 null 的 HashMap。Set 类型的数据具有无序、元素不可重复、查找速度快和支持交集、并集、差集等功能的特征，常见命令如下。

- SADD key member1 [member2 ...]：将一个或多个 member 元素加入到集合 key 中。
- SREM key member1 [member2 ...]：删除集合 key 中的一个或多个 member 元素。
- SCARD key：返回集合 key 中元素的个数。
- SISMEMBER key member：判断元素 member 是否存在于集合 key 中。
- SMEMBERS key：获取集合 key 中的所有元素。
- SINTER key1 [key2 ...]：获取所有集合的交集。

（4）SortedSet

Redis 中的 SortedSet 是一个可排序的 Set 集合，与 Java 中的 TreeSet 有些类似，但底层数据结构差别很大。SortedSet 中的每一个元素都带有一个 score 属性，可以基于 score 属性对元素排序，底层的实现是一个跳表（SkipList）加 Hash 表。

SortedSet 类型的数据具有可排序、元素不重复、查询速度快的特性，常用来实现排行榜这样的功能。SortedSet 常见命令如下。

- ZADD key score member：添加一个或多个元素到集合 key 中 ，如果已经存在则更新其 score 值。
- ZREM key member：删除集合 key 中的元素 member。
- ZSCORE key member：获取集合 key 中的元素 member 的 score 值。
- ZRANK key member：获取集合 key 中的元素 member 的排名。
- ZCARD key：获取集合 key 中的元素个数。
- ZCOUNT key min max：统计集合 key 中 score 值在给定范围内的所有元素的个数。

- ZINCRBY key increment member：让集合 key 中的元素 member 根据指定步长 increment 自增。

（5）Hash

Hash 是由字符串类型的 field 和 Value 组成的映射表，可以把它理解成一个包含了多个键值对的集合，一般用于存储对象。Hash 有以下常见命令。

- HSET key field value：将集合 key 中字段 field 的值设为 value。
- HGET key field：获取集合 key 中字段 field 的值。
- HMSET key field1 value1[field2 value2 ...]：将一个或多个 field-value（字段-值）对设置到集合 key 中。
- HMGET key field1 [field2 ...]：获取集合 key 中一个或多个 field 的值。
- HGETALL key：获取集合 key 中的所有的 field 和 Value。
- HKEYS key：获取集合 key 中的所有的 field。
- HINCRBY key field increment：让集合 key 中的 field 字段根据指定步长 increment 自增。
- HSETNX key field value：在集合 key 中，当且仅当 field 不存在时，添加字段 field，字段对应的值为 value。

Redis 所包含的知识比较多，在此只对其概念、数据类型和常见命令进行了简单讲解。

5.4.2　Spring Data Redis 快速入门

为了方便开发者使用 Redis，Redis 官方为主流的编程语言都提供了对应的客户端，其中面向 Java 的客户端有 Redisson、Jedis 和 Lettuce 等。Jedis 和 Lettuce 提供了 Redis 命令对应的 API，因此操作 Redis 比较方便。如果一个项目中使用了 Lettuce 连接 Redis，后来决定弃用 Lettuce 改用 Jedis，就要面临修改代码的问题，对于此种问题，可以使用 Spring Data Redis。下面对 Spring Data Redis 概述和常见操作进行讲解。

1. Spring Data Redis 概述

Spring Data Redis 是 Spring Data 在 Spring 管理的项目中对 Redis 操作的具体实现，Spring Data Redis 提供了 Redis 面向 Java 语言客户端的抽象，在开发中可以忽略掉切换具体的客户端所带来的影响，可实现自动管理。Spring Boot 为支持 Redis 提供了 spring-boot-starter-data-redis.jar，该 Starter 使用 Spring Data Redis 对底层 Lettuce 和 Jedis 进行了封装，并为 Lettuce 和 Jedis 提供了自动配置。Spring Data Redis 具有以下特性。

- 提供了对不同 Redis 客户端的整合（Lettuce 和 Jedis）。
- 提供了 RedisTemplate 统一 API 来操作 Redis。
- 支持 Redis 的发布订阅模型。
- 支持 Redis 哨兵和 Redis 集群。
- 支持基于 Lettuce 的响应式编程。
- 支持基于 JDK、JSON、字符串、Spring 对象的数据序列化和反序列化。
- 支持基于 Redis 的 JDK Collection 实现。

Spring Data Redis 是 Spring Data 的具体实现，因此 Spring Data Redis 也具备 Spring Data 的特性。使用 Spring Data Redis 时，项目的 DAO 接口只需继承 CrudRepository，Spring Data

Redis 能为 DAO 组件提供实现类。Spring Data Redis 也支持方法名关键字查询，只不过 Redis 查询的属性必须是被索引过的，而且 Spring Data Redis 支持的方法名关键字查询功能不如 JPA 强大，只支持 And、Or、Is、Equals 等简单的关键字，而不支持 LessThan、Like 等复杂关键字。

2. Spring Data Redis 常见操作

（1）RedisTemplate 常见 API

Spring Data Redis 中提供了 RedisTemplate 工具类，该工具类封装了各种对 Redis 的操作，并且将不同数据类型的操作 API 封装到了不同的 Operation 接口对象中，获取常用 Operation 接口对象的方法如表 5-2 所示。

表 5-2 获取常用 Operation 接口对象的方法

方法	说明
ValueOperations<K, V> opsForValue()	获取操作 String 类型数据的对象
ListOperations<K, V> opsForList()	获取操作 List 类型数据的对象
SetOperations<K, V> opsForSet()	获取操作 Set 类型数据的对象
ZSetOperations<K, V> opsForZSet()	获取操作 SortedSet 类型数据的对象
HashOperations<K, HK, HV> opsForHash()	获取操作 Hash 类型数据的对象

RedisTemplate 对 Redis 命令进行了分类，不同的命令由不同的接口提供支持，例如，操作 List 的命令由 ListOperations 负责提供；操作 Set 的命令由 SetOperations 负责提供；RedisTemplate 提供对应的方法返回相应的操作接口对象。

为了能更便捷地操作 Redis，RedisTemplate 还可以通过 bound 绑定指定的 Key，绑定 Key 后再次进行一系列的操作时，无须再次指定 Key。常用的绑定 Key 的方法如表 5-3 所示。

表 5-3 常用的绑定 Key 的方法

方法	说明
BoundValueOperations<K, V> boundValueOps(K key)	绑定映射 String 类型数据的 Key
BoundListOperations<K, V> boundListOps(K key)	绑定映射 List 类型数据的 Key
BoundSetOperations<K, V> boundSetOps(K key)	绑定映射 Set 类型数据的 Key
BoundZSetOperations<K, V> boundZSetOps(K key)	绑定映射 SortedSet 类型数据的 Key
BoundHashOperations<K, HK, HV> boundHashOps(K key)	绑定映射 Hash 类型数据的 Key

（2）Spring Data Redis 的常用注解和序列化策略

Spring Data Redis 操作的不是持久化类而是数据类，为了实现数据类与 Redis 之间的映射关系，Spring Data Redis 提供了以下注解。

- @RedisHash：该注解在类上进行标注，用于指定将数据类映射到 Redis 中的存储空间。
- @Id：用于标识实体类主键。在 Redis 数据库中会默认生成字符串形式的 HashKey，用于标识唯一的实体对象 id，当然也可以在数据存储时手动指定 id。
- @Indexed：用于标示对应属性在 Redis 数据库中生成的二级索引。使用该注解后会在 Redis 数据库中生成属性对应的二级索引，索引名称就是属性名，可以方便地进行数据

条件查询。

Redis 的序列化策略有两种，一种是 String 的序列化策略，另一种是 JDK 的序列化策略。RedisTemplate 默认采用的是 JDK 的序列化策略，保存的 Key 和 Value 都采用此策略进行序列化保存。RedisTemplate 使用 JDK 的序列化策略保存数据时，会将数据先序列化成字节数组，然后再存入 Redis 数据库，如果使用 Redis 查看对应的数据，数据是以字节数组显示的，而不是以原生可读的形式显示的。

RedisTemplate 有一个子类 StringRedisTemplate，StringRedisTemplate 默认采用的是 String 的序列化策略，保存的 key 和 value 都是采用此策略序列化保存的，默认存入的数据就是原始的字符串。

5.4.3　整合 Redis

通过前面的学习，读者对 Spring Data Redis 有了基本的了解。下面将 Spring Boot 和 Redis 进行整合，进一步演示在 Spring Boot 项目中使用 Spring Data Redis 操作 Redis 的基本使用，具体如下。

1. 配置依赖

在项目 chapter05 的 pom.xml 文件中配置 Spring Boot 提供的 Spring Data Redis 启动器依赖，具体代码如下。

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-redis</artifactId>
</dependency>
```

2. 设置配置信息

引入的 spring-boot-starter-data-redis 依赖，提供了操作 Redis 的一些默认配置信息。例如，连接的 Redis 主机地址、主机的端口号、指定连接的数据库、连接池的连接数等信息。如果没有指定会默认连接本地端口为 6379 的主机。本案例演示的 Redis 安装在本地，所以不用进行额外的配置，使用默认的配置信息即可。

3. 创建数据类

在 com.itheima.chapter05.entity 包下创建数据类 User，具体如文件 5-15 所示。

文件 5-15　User.java

```
1  import org.springframework.data.annotation.Id;
2  import org.springframework.data.redis.core.RedisHash;
3  import org.springframework.data.redis.core.index.Indexed;
4  @RedisHash("user")
5  public class User {
6     @Id
7     private Integer id;
8     @Indexed
9     private  String name;
10   //setter/getter 方法，以及 toString()方法
11 }
```

在上述代码中，第 4 行代码使用@RedisHash 指定当前数据类的实例对象的数据，将其存储在 Redis 数据库中名为 user 的存储空间下；第 6 行代码使用@Id 声明当前属性为标识属性，@Id 为 Spring Data 提供的注解；第 8 行代码使用@Indexed 声明当前属性会被“索引

化”，存储在 Redis 时会为当前属性创建对应的 Key。

4. 自定义 Repository 接口

在 com.itheima.chapter05.dao 包下创建自定义接口 UserRepository，并使用该接口继承 CrudRepository 接口，具体如文件 5-16 所示。

文件 5-16 UserRepository.java

```
1  import com.itheima.chapter05.entity.User;
2  import org.springframework.data.repository.CrudRepository;
3  import org.springframework.stereotype.Repository;
4  @Repository
5  public interface UserRepository extends CrudRepository<User,Integer> {
6  }
```

在上述代码中，第 5 行代码指定操作的数据类为 User，并且没有自定义方法。

5. 定义测试方法

在 src\test\java 目录的 com.itheima.chapter05 包下创建测试类 Chapter05ApplicationRedisTests，在该测试类中定义操作用户信息的测试方法，具体如文件 5-17 所示。

文件 5-17 Chapter05ApplicationRedisTests.java

```
1  import com.itheima.chapter05.dao.UserRepository;
2  import com.itheima.chapter05.entity.User;
3  import org.junit.jupiter.api.Test;
4  import org.springframework.beans.factory.annotation.Autowired;
5  import org.springframework.boot.test.context.SpringBootTest;
6  import org.springframework.data.redis.core.*;
7  import java.util.List;
8  import java.util.Map;
9  import java.util.Set;
10 @SpringBootTest
11 class Chapter05ApplicationRedisTests {
12     @Autowired
13     private UserRepository userRepository;
14     @Autowired
15     private StringRedisTemplate stringRedisTemplate;
16     @Test
17     void saveTest() {
18         User user = new User(1,"zhangsan");
19         userRepository.save(user);
20         Optional<User> u = userRepository.findById(1);
21         System.out.println(u);
22     }
23     @Test
24     void stringTest() {
25         //存入 String 数据
26         stringRedisTemplate.boundValueOps("name").set("lisi");
27         //获取 String 数据
28         String name = stringRedisTemplate.boundValueOps("name").get();
29         System.out.println("name = " + name);
30     }
31     @Test
32     void listTest() {
```

```
33        //绑定映射 List 类型数据的 Key
34        BoundListOperations<String, String> hobby =
35          stringRedisTemplate.boundListOps("hobby");
36        //在绑定的键中添加值
37        hobby.leftPush("swim");
38        hobby.leftPush("travel");
39        //获取绑定键中的值
40        List<String> hbs = hobby.range(0, -1);
41        System.out.println(hbs);
42    }
43    @Test
44    void setTest() {
45        //绑定映射 Set 类型数据的 Key
46        BoundSetOperations<String, String> subject =
47          stringRedisTemplate.boundSetOps("subject");
48        //添加元素
49        subject.add("Chinese");
50        subject.add("English");
51        //获取
52        Set<String> members = subject.members();
53        System.out.println(members);
54    }
55    @Test
56    void hashTest() {
57        //绑定映射 Hash 类型数据的 Key
58        BoundHashOperations<String, Object, Object> role =
59          stringRedisTemplate.boundHashOps("role");
60        //添加元素
61        role.put("admin","wangwu");
62        role.put("user","zhaoliu");
63        Map<Object, Object> entries = role.entries();
64        System.out.println(entries);
65    }
66 }
```

在上述代码中，第 12～15 行代码注入了 UserRepository 对象和 StringRedisTemplate 对象；第 17～65 行代码定义了 5 个方法。其中，第 17～22 行代码的 saveTest()方法中分别调用 UserRepository 的 save()方法和 findById()方法将 User 对象保存到 Redis 中，保存后通过用户 id 获取对应的用户信息，save()方法和 findById()方法都继承自 CrudRepository。

除了可以使用继承自 CrudRepository 的方法操作用户信息外，还可以使用 StringRedisTemplate 工具类在 Redis 中写入或读取数据。其中，第 24～30 行代码的 stringTest()方法用于测试操作 String 类型的数据；第 32～42 行代码的 listTest()方法用于测试操作 List 类型的数据；第 44～54 行代码的 setTest()方法用于测试操作 Set 类型的数据；第 56～65 行代码的 hashTest()方法用于测试操作 Hash 类型的数据。这些方法都包含存入和获取对应类型数据的操作。

6. 测试操作图书信息

启动 Redis 服务后，运行文件 5-17 中的 saveTest()方法，测试保存和查询用户信息，控制台输出如图 5-21 所示。

从图 5-21 可以看出，控制台输出的用户信息与保存的时候一致，说明成功将用户信息

保存在 Redis 中，并可从 Redis 中查询到对应的用户信息。

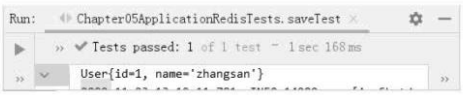

图5-21 测试保存和查询用户信息

启动 RESP.app，查看此时 Redis 中存储的数据，如图 5-22 所示。

图5-22 Redis中存储的数据

从图 5-22 可以看出，虽然程序只保存了一个 User 对象，但 Redis 中保存了多对键值对，由于前面在 User 类上增加了@RedisHash("user")注解，因此这些 Key 的名字都以 user 开头。其中，名称为 user 的 key 对应一个 Set，该 Set 中的元素就是 User 对象的标识属性值。由于此时系统中仅有一个 User 对象，因此该 Key 对应的 Set 中只有一个元素。

单击图 5-22 名称为“user:1”的 Key，内容如图 5-23 所示。

图5-23 Key为“user:1”的数据

在图 5-23 中，名称为“user:1”的 Key 中的 1 为标识属性值，该 Key 所对应的是一个 Hash，它完整地保存了整个 User 对象的所有数据，当程序要根据 id 获取某个 Book 对象时，Redis 直接获取 Key 为“book:id 值”的 Value，这样就得到了该 User 对象的全部数据。

User 类中的 name 属性使用@Indexed 注解进行标注，因此保存对应的 User 对象时，会为对应的属性创建对应的 Key。在图 5-23 中，单击名称为“user:name:zhangsan”的 Key，

内容如图 5-24 所示。

图5-24　Key为“ user:name:zhangsan ”的数据

从图 5-24 可以看出，“ user:name:zhangsan ”的 Key 对应的是一个 Set，该 Set 的成员就是 User 对象的 id 属性的值。因为当程序保存多个 User 对象时，完全有可能多个 User 对象的 name 属性值都是“ zhangsan ”，此时这些对象的 id 都需要由名称为“ user:name:zhangsan ”的 Key 所对应的 Set 负责保存，因此该 key 对应的是一个 Set。

名称为“ user:1:idx ”的 Key 对应的 Value 中，保存了该 User 对象所有额外的 Key，其中，1 为标识属性值。在图 5-24 中，单击名称为“ user:1:idx ”的 Key，内容如图 5-25 所示。

从图 5-25 可以看出，名称为“ user:1:idx ”的 Key 对应的 Value 也是 Set。

图5-25　Key为“ user:1:idx ”的数据

运行文件 5-17 中的 stringTest()方法，测试保存和查询 String 类型的数据，控制台输出如图 5-26 所示。

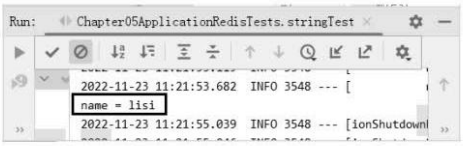

图5-26　保存和查询String类型的数据

从图 5-26 可以看出，控制台输出的信息与保存的时候一致，说明成功将 String 类型的数据保存在 Redis 中，并可根据 Key 查询出对应的信息。

在 RESP.app 中查看 Key 为 name 的数据，如图 5-27 所示。

从图 5-27 可以看出，Key 为 name 的数据对应的 Value 是 String 类型，Value 的值为 lisi，说明 stringTest()方法保存数据成功。

运行文件 5-17 中的 listTest()方法，测试保存和查询 List 类型的数据，控制台输出如图 5-28 所示。

从图 5-28 可以看出，控制台输出的信息与保存的时候一致，说明成功将 List 类型的数据保存在 Redis 中，并查询出了 List 中指定范围的数据。

图5-27 查看Key为name的数据

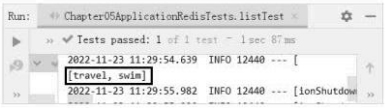

图5-28 保存和查询List类型的数据

在 RESP.app 中查看 Key 为 hobby 的数据，如图 5-29 所示。

从图 5-29 可以看出，Key 为 hobby 的数据对应的 Value 是 List 类型，Value 中的元素为 travel 和 swim，说明 listTest()方法保存数据成功。

运行文件 5-17 中的 setTest()方法，测试保存和查询 Set 类型的数据，控制台输出如图 5-30 所示。

图5-29 查看Key为hobby的数据

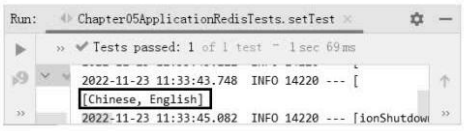

图5-30 保存和查询Set类型的数据

从图 5-30 可以看出，控制台输出的信息与保存的时候一致，说明成功将 Set 类型的数据保存在 Redis 中，并查询出了 Set 中所有的数据。

在 RESP.app 中查看 Key 为 subject 的数据，如图 5-31 所示。

从图 5-31 可以看出，Key 为 subject 的数据对应的 Value 是 Set 类型，Value 中的元素为 Chinese 和 English，说明 setTest()方法保存数据成功。

运行文件 5-17 中的 hashTest()方法，测试保存和查询 Hash 类型的数据，控制台输出如图 5-32 所示。

从图 5-32 可以看出，控制台输出的信息与保存的时候一致，说明成功将 Hash 类型的数据保存在 Redis 中，并查询出了 Hash 中所有的数据。

图5-31　查看Key为subject的数据

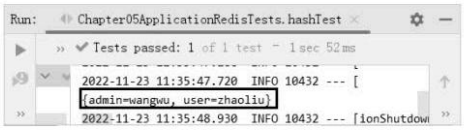

图5-32　保存和查询Hash类型的数据

在 RESP.app 中查看 Key 为 role 的数据，如图 5-33 所示。

图5-33　查看Key为role的数据

从图 5-33 可以看出，Key 为 role 的数据对应的 Value 是 Hash 类型，Value 中的元素为两对键值对，说明 hashTest()方法保存数据成功。

至此，Spring Boot 整合 Redis 后，在项目中演示 Spring Data Redis 的基本使用已经完成。

5.5　本章小结

本章主要对 Spring Boot 数据访问进行了讲解。首先讲解了 Spring Data；然后讲解了 Spring Data JPA 的基础知识，以及 Spring Boot 整合 Spring Data JPA，接着讲解了 MyBatis-Plus 的基础知识，以及 Spring Boot 整合 MyBatis-Plus。最后讲解了 Redis 和 Spring Data Redis 的基础知识，以及 Spring Boot 整合 Redis。通过本章的学习，希望大家可以在 Spring Boot 项目中正确应用数据访问技术，为后续更深入地学习 Spring Boot 做好铺垫。

5.6　本章习题

一、填空题

1．Spring Data JPA 默认使用_____实现 JPA。

2．_____注解可以扫描指定路径的 Mapper 并交由 Spring 管理。

3．Redis 默认启动端口号为_____。

4．Redis 的序列化策略有 String 的序列化策略和_____的序列化策略。

5．使用@Indexed 声明当前属性会被“索引化”，存储在 Redis 时会为当前属性创建对

应的_____。

二、判断题

1．Spring Data 为大量的关系型数据库和非关系型数据库提供了数据访问的方案。（　　）

2．Spring Data JPA 使用位置参数时，在@Query 注解的查询语句中，对参数标注的编号和方法的参数列表中参数的顺序的对应关系没有要求。（　　）

3．MyBatis-Plus 在 MyBatis 的基础上只做增强不做改变。（　　）

4．Redis 以 Key-Value 的形式存储数据。（　　）

5．@RedisHash 可以指定数据类的实例对象存储在 Redis 存储空间的名称。（　　）

三、选择题

1．下列选项中，关于 Spring Data 的接口描述错误的是（　　）。

A．Repository 接口提供了各种增删改查方法。

B．CrudRepository 接口继承自 Repository 接口，提供了各种增删改查方法。

C．PagingAndSortingRepository 接口继承自 CrudRepository 接口，增加了分页查询和排序查询的方法。

D．QueryByExampleExecutor 接口是进行条件封装查询的顶级父接口。

2．下列选项中，关于 JPQL 的使用描述错误的是（　　）。

A．JPQL 支持命名参数和位置参数两种查询参数。

B．使用 JPQL 时，需要把查询的表名换成实体类名称，把表中的字段名换成实体类的属性名称。

C．JPQL 中可以使用 SpEL 表达式接收变量。

D．JPQL 中只能使用字符串和基本数据类型的数据作为参数。

3．下列选项中，关于 Wrapper 的 eq()方法的作用描述正确的是（　　）。

A．用于匹配字段中值不等于某个值的记录。

B．用于匹配字段中值大于某个值的记录。

C．用于匹配字段中值等于某个值的记录。

D．用于匹配字段中值大于或等于某个值的记录。

4．下列选项中，使用 MyBatis-Plus 时，可以指定实体类中属性对应字段名的注解是（　　）。

A．@Entity　　B．@TableName　　C．@TableId　　D．@TableField

5．下列选项中，Spring Data Redis 用于标示对应属性在 Redis 数据库中生成二级索引的注解是（　　）。

A．@RedisHash　B．@Id　　C．@Indexed　　D．@TableField

第6章

Spring Boot整合缓存

学习目标

★ 了解 Spring Boot 默认缓存方案，能够说出默认缓存方案的执行流程

★ 掌握声明式缓存注解，能够说出@EnableCaching、@Cacheable、@CachePut、@CacheEvict、@Caching、@CacheConfig 注解及常用属性的作用

★ 掌握声明式缓存注解的应用，能够在 Spring Boot 项目中正确应用声明式缓存注解

★ 了解 Ehcache 概述，能够说出 Ehcache 的特点

★ 掌握整合 Ehcache，能够在 Spring Boot 项目中整合 Ehcache，并正确应用声明式缓存注解

★ 掌握 SpringBoot 整合 Redis 缓存，能够在 Spring Boot 项目中整合 Redis 缓存，并正确应用声明式缓存注解

企业级应用为了避免读取数据时受限于数据库的访问效率而导致整体系统性能偏低，通常会在应用程序与数据库之间建立一种临时的数据存储机制，该临时存储数据的区域称为缓存。缓存是一种介于数据永久存储介质与应用程序之间的数据临时存储介质，可以提供临时的数据存储空间，合理使用缓存可以有效减少低速数据读取（例如磁盘 IO）过程的次数，以提高系统性能。Spring Boot 提供了几乎所有主流缓存技术的整合方案。下面将对 Spring Boot 整合常见的缓存技术进行讲解。

6.1 Spring Boot 默认缓存管理

Spring框架支持透明地向应用程序添加缓存，以及对缓存进行管理，其管理缓存的核心是将缓存应用于操作数据的方法，从而减少操作数据的执行次数，同时不会对程序本身造成任何干扰。Spring Boot 继承了 Spring 框架的缓存管理功能，下面将对 Spring Boot 默认缓存方案和声明式缓存注解进行讲解。

6.1.1 Spring Boot 默认缓存方案

Spring 的缓存机制将提供的缓存作用于 Java 方法上，基于缓存中的可用信息，可以减

少方法的执行次数。每次目标方法调用时，抽象使用缓存行为来检查执行方法，即检查执行方法是否给定了缓存的执行参数，如果是，则返回缓存结果，不执行具体方法；如果否，则执行方法，并将结果缓存后，返回给用户。

Spring默认的缓存方案通过 org.springframework.cache.Cache 和 org.springframework.cache.CacheManager 接口来统一不同的缓存技术，其中 Cache 接口为缓存的组件定义了规范，包含缓存的各种操作集合。Spring 根据 Cache 接口中定义的缓存规则提供了多种实现缓存机制的具体方式和技术，这些具体方式和技术通常称为缓存实现，常见的缓存实现有 RedisCache、EhCache、ConcurrentMapCache 等。CacheManager 是缓存管理器，其基于缓存名称对缓存进行管理，并制定了管理 Cache 的规则。

在项目中添加某个缓存管理组件（如 Redis）后，Spring Boot 项目会选择并启用对应的缓存管理器。如果项目中同时添加了多个缓存组件，且没有定义类型为 CacheManager 的 Bean 组件或者名称为 cacheResolver 的缓存解析器，Spring Boot 将尝试按以下列表的顺序查找有效的缓存组件进行缓存管理。

① Generic。
② JCache（EhCache 3、Hazelcast、Infinispan 等）。
③ EhCache 2.x。
④ Hazelcast。
⑤ Infinispan。
⑥ Couchbase。
⑦ Redis。
⑧ Caffeine。
⑨ Simple。

在项目中开启缓存管理后，如果没有任何缓存组件，会默认使用最后一个 Simple 缓存组件进行管理。Simple 缓存组件是 Spring Boot 默认的缓存管理组件，它默认使用内存中的 ConcurrentHashMap 进行缓存存储，所以在没有添加任何第三方缓存组件的情况下，可以实现内存中的缓存管理。

6.1.2 声明式缓存注解

要想使用 Spring 提供的默认缓存，需要对缓存进行声明，也就是声明缓存的方法和缓存策略。对于缓存声明，Spring 提供了一系列的注解，使用这些注解可以实现 Spring 默认的基于注解的缓存管理，下面将对这些缓存注解进行详细讲解。

1. @EnableCaching 注解

@EnableCaching 是 Spring 框架提供的用于开启基于注解的缓存支持的注解，当配置类上使用了@EnableCaching 注解，会默认提供实现 CacheManager 的缓存组件，并通过 AOP（Aspect Oriented Programming，面向切面编程）将缓存行为添加到应用程序。执行操作时，会检查是否已经存在注解对应的缓存，如果找到了，就会自动创建一个代理来拦截方法调用，使用缓存的 Bean 执行处理。

2. @Cacheable 注解

@Cacheable 注解用于标注可缓存的方法，通常标注的方法为数据查询方法。标注@Cacheable

注解的方法在执行时，会先查询缓存，如果查询到的缓存为空，则执行该方法，并将方法的执行结果添加到缓存；如果查询到缓存数据，则不执行该方法，而是直接使用缓存数据。

@Cacheable 注解提供了多个属性，用于对缓存进行相关配置，具体属性及说明如表 6-1 所示。

表 6-1　@Cacheable 注解属性及说明

属性	说明
value/cacheNames	指定缓存的名称，必备属性，这两个属性二选一使用
key	指定缓存数据的 key，默认使用方法参数值，可以使用 SpEL 表达式
keyGenerator	指定缓存数据的 key 的生成器，与 key 属性二选一使用
cacheManager	指定缓存管理器
cacheResolver	指定缓存解析器，与 cacheManager 属性二选一使用
condition	指定在符合某些条件下进行数据缓存
unless	指定在符合某些条件下不进行数据缓存
sync	指定是否使用异步缓存，默认为 false

下面对@Cacheable 注解的属性进行具体讲解。

（1）value/cacheNames 属性

value 和 cacheNames 属性作用相同，用于指定缓存的名称，方法的返回结果会存放在指定名称的缓存中。这两个属于必备选项，且要二选一使用。如果@Cacheable 注解只配置 value 或者 cacheNames 属性，那么属性名可以省略，示例代码如下。

```
@Cacheable("book")
public Book findById(Integer id){
    return bookDao.findById(id).get();
}
```

在上述代码中，指定了缓存的名称为 book，当第一次执行 findById()方法时，会将该方法的返回结果存储在名称为 book 的缓存中。

@Cacheable 注解中可以指定多个缓存的名称，以便使用多个缓存，示例代码如下。

```
@Cacheable({"book","hotBook"})
public Book findById(Integer id){
    return bookDao.findById(id).get();
}
```

在上述代码中，指定了 book 和 hotBook 两个缓存的名称。如果同时指定多个缓存名称，在缓存中查找数据时，如果在任意一个缓存中存在请求所需的数据，则返回缓存中相关的值。

（2）key 属性

缓存的本质是键值对存储，key 用于指定唯一的标识，value 用于指定缓存的数据，所以每次调用缓存方法都会转换为访问缓存的键。缓存的键通过 key 属性进行指定，进行数据缓存时，如果没有指定 key 属性，Spring Boot 默认配置类 SimpleKeyGenerator 中的 generate Key(Object... params)方法会根据方法参数生成 key 值。对于没有参数的方法，其 key 是默认创建的空参 SimpleKey[]对象；对于有一个参数的方法，其 key 是默认参数值；对于有多个参数的方法，其 key 是包含所有参数的 SimpleKey 对象。

如果方法有多个参数，但是部分参数对缓存没有任何用处，通常会选择手动指定 key 属性的值，key 属性的值可以通过 SpEL 表达式选择，示例代码如下。

```
@Cacheable(cacheNames="book", key="#id")
public Book findBookById(Integer id, boolean includeUsed){
    return bookDao.findById(id).get();
}
```

在上述代码中，指定缓存的 key 为 findBookById()方法中的参数 id。

每个 SpEL 表达式在求值时都会关联到一个上下文，在对应的上下文中可以使用 SpEL 表达式进行键和条件的计算，Cache 缓存支持的 SpEL 表达式及说明如表 6-2 所示。

表 6-2 Cache 缓存支持的 SpEL 表达式及说明

参数	位置	描述	示例
methodName	root 对象	当前被调用的方法名	#root.methodName
method	root 对象	当前被调用的方法	#root.method.name
target	root 对象	当前被调用的目标对象实例	#root.target
targetClass	root 对象	当前被调用的目标对象的类	#root.targetClass
args	root 对象	当前被调用的方法的参数列表	#root.args[0]
caches	root 对象	当前被调用的方法的缓存列表	#root.caches[0].name
argument Name	执行上下文	当前被调用的方法参数，可以用#参数名或者#a0、#p0 的形式（0 代表参数索引，从 0 开始）	#comment_id、#a0、#p0
result	执行上下文	当前方法执行后的返回结果	#result

（3）keyGenerator 属性

keyGenerator 属性与 key 属性本质作用相同，都是用于指定缓存数据的 key，只不过 keyGenerator 属性指定的不是具体的 key 值，而是 key 值的生成器规则，由其中指定的生成器生成具体的 key。使用时，keyGenerator 属性与 key 属性要二者选一。关于自定义 key 值生成器的定义，可以参考 Spring Boot 默认配置类 SimpleKeyGenerator 的定义方式，这里不再做具体说明。

（4）cacheManager/cacheResolver 属性

cacheManager 和 cacheResolver 属性分别用于指定缓存管理器和缓存解析器，这两个属性也是二选一使用，默认情况下不需要配置，对于需要使用多个缓存管理器（如 Redis、Ehcache 等）的应用，可以为每个操作设置一个缓存管理器或缓存解析器。

（5）condition 属性

condition 属性用于对数据进行有条件的选择性存储，只有当指定条件为 true 时才会对查询结果进行缓存，可以使用 SpEL 表达式指定属性值，示例代码如下。

```
@Cacheable(cacheNames="book", condition="#id > 1")
public Book findBook(Integer id){
    return bookDao.findById(id).get();
}
```

在上述代码中，只有当参数 id 大于 1 的时候，才会对方法的结果进行缓存。

（6）unless 属性

unless 属性的作用与 condition 属性相反，当指定的条件为 true 时，方法的返回值不会被缓存，也可以使用 SpEL 表达式指定，示例代码如下。

```
@Cacheable(cacheNames="book", unless = "#result==null")
public Book findBook(Integer id){
```

```
    return bookDao.findById(id).get();
}
```

在上述代码中，unless 属性指定只有查询结果不为空时才会对结果数据进行缓存。

（7）sync 属性

在多线程程序中，某些操作可能会同时引用相同的参数，导致相同的对象被计算好几次，从而达不到缓存的目的。对于这种情况，可以使用 sync 属性，sync 属性表示数据缓存过程中是否使用同步模式，默认值为 false，通常不会使用该属性。

3. @CachePut 注解

@CachePut 注解的作用是更新缓存数据，当需要更新缓存且不影响方法执行时，可以使用@CachePut 注解，通常用在数据更新方法上。@CachePut 注解的执行顺序是，先进行方法调用，然后将方法结果更新到缓存中。

@CachePut 注解也提供了多个属性，这些属性与@Cacheable 注解的属性完全相同。通常不建议在同一个方法中同时使用@CachePut 和@Cacheable 注解，这两个注解关注不同的行为，@CachePut 注解会强制执行方法并进行缓存更新，使用@Cacheable 注解时，如果请求能够在缓存中获取到对应的数据，就不会执行当前被@Cacheable 注解标注的方法。读者也应根据需求场景选择对应的注解进行声明。

4. @CacheEvict 注解

@CacheEvict 注解的作用是删除缓存中的数据，通常标注在数据删除方法上。@CacheEvict 注解的默认执行顺序是先进行方法调用，然后将缓存清除。

@CacheEvict 注解也提供了多个属性，这些属性与@Cacheable 注解的属性基本相同，除此之外，还额外提供了两个特殊属性 allEntries 和 beforeInvocation，下面对这两个属性分别进行讲解。

（1）allEntries 属性

allEntries 属性表示是否清除指定缓存空间中的所有缓存数据，默认值为 false，即默认只删除指定 key 对应的缓存数据，示例代码如下。

```
@CacheEvict(cacheNames = "book",allEntries = true)
public void delById(Integer id){
    bookDao.deleteById(id);
}
```

在上述代码中，通过 allEntries 属性指定方法执行后会删除名称为 book 的缓存中所有的数据。

（2）beforeInvocation 属性

beforeInvocation 属性表示是否在方法执行之前进行缓存清除，默认值为 false，即默认在执行方法后再进行缓存清除，示例代码如下。

```
@CacheEvict(cacheNames = "book",beforeInvocation = true)
public void delById(Integer id){
    bookDao.deleteById(id);
}
```

在上述代码中，使用 beforeInvocation 属性指定在执行 delById()方法之前将缓存 book 中的数据清除。

虽然可以将 beforeInvocation 属性的值设置为 true，以实现在方法执行前清除缓存，但这样做也存在一定的弊端。例如，删除数据的方法在执行的过程中发生了异常，导致实际数据并没有被删除，但是缓存数据却被提前清除了。

5. @Caching 注解

如果不同缓存之间的条件或者键表达式不同，就需要指定相同类型的多个注解，例如需要同时指定多个@CacheEvict 或@CachePut，这个时候可以使用@Caching 注解。@Caching 注解用于对复杂规则的数据进行缓存管理，@Caching 注解中允许使用多个嵌套的 @Cacheable、@CachePut 或 @CacheEvict。在@Caching 注解内部包含有 Cacheable、put 和 evict 三个属性，分别对应于@Cacheable、@CachePut 和@CacheEvict 三个注解，示例代码如下。

```
@Caching(evict = { @CacheEvict("primary"),
@CacheEvict(cacheNames="secondary", key="#date")})
public void delById(Integer id, Date date){
    bookDao.deleteById(id);
}
```

在上述代码中，delById()方法使用了@Caching 注解，并使用 evict 属性嵌套引入了两个 @CacheEvict 注解，分别用于删除名称为 primary 和 secondary 的缓存，其中删除 secondary 缓存中数据的时候应根据指定的 key 进行删除。

6. @CacheConfig 注解

@CacheConfig 注解使用在类上，主要用于统筹管理类中所有使用@Cacheable、@CachePut 和@CacheEvict 注解标注方法中的公共属性，这些公共属性包括 cacheNames、keyGenerator、cacheManager 和 cacheResolver，示例代码如下。

```
@CacheConfig(cacheNames = "book")
@Service
public class BookService {
    @Autowired
    private BookRepository bookRepository;
    @Cacheable
    public Book findById(Integer id){
        return  bookRepository.findById(id).get();
    }
}
```

在上述代码中，BookService 类上标注了@CacheConfig 注解，同时使用 cacheNames 属性将缓存空间统一设置为 book，这样在该类中所有方法上使用缓存注解时可以省略相应的 cacheNames 属性，默认为 book。

需要说明的是，如果在类上使用了@CacheConfig 注解定义了某个属性，同时又在该类方法中使用缓存注解定义了相同的属性，那么该属性值会使用“就近原则”选择以方法的注解中的属性值为准。

6.1.3 声明式缓存注解的应用

声明式缓存注解是一个重要的开发工具，可以方便地实现缓存控制，帮助程序提升执行效率。声明式缓存注解的编写过程中，应该根据实际需求进行声明式缓存的设计，因地制宜，灵活运用缓存技术，避免“重缓存、轻逻辑”的误区，努力提升程序运行的效率和可用性。同时，开发者应该将缓存技术科学地应用于实际场景中，使其为用户带来更好的使用体验。下面在 Spring Boot 项目中进一步对声明式缓存注解的应用进行讲解，具体如下。

1. 创建项目

在 IDEA 中创建 Spring Boot 项目 chapter06，读者可以根据自己当前情况选择使用 Spring

Initializr 方式或者 Maven 方式进行创建，在此选择使用 Maven 方式创建项目。

2. 配置依赖

为了更好地演示应用缓存后的效果，创建的 Spring Boot 项目中整合了 Spring MVC、Spring Data JPA、MySQL。在项目 chapter06 的 pom.xml 文件中配置 Spring MVC、Spring Data JPA、MySQL，以及 Spring Boot 提供的缓存启动器的依赖，具体如文件 6-1 所示。

文件 6-1　pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">
   <modelVersion>4.0.0</modelVersion>
   <parent>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-parent</artifactId>
      <version>2.7.4</version>
      <relativePath/>
   </parent>
   <groupId>com.itheima</groupId>
   <artifactId>chapter06</artifactId>
   <version>0.0.1-SNAPSHOT</version>
   <name>chapter06</name>
   <description>chapter06</description>
   <properties>
      <java.version>11</java.version>
   </properties>
   <dependencies>
      <dependency>
         <groupId>org.springframework.boot</groupId>
         <artifactId>spring-boot-starter-cache</artifactId>
      </dependency>
      <dependency>
         <groupId>org.springframework.boot</groupId>
         <artifactId>spring-boot-starter-web</artifactId>
      </dependency>
      <dependency>
         <groupId>org.springframework.boot</groupId>
         <artifactId>spring-boot-starter-data-jpa</artifactId>
      </dependency>
      <dependency>
         <groupId>mysql</groupId>
         <artifactId>mysql-connector-java</artifactId>
      </dependency>
   </dependencies>
   <build>
      <plugins>
         <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
```

```
            </plugin>
        </plugins>
    </build>
</project>
```

3. 设置配置信息

在项目的 resources 目录下创建 application.yml 文件，在该文件中指定数据库连接信息和 JPA 的配置信息，具体如文件 6-2 所示。

文件 6-2 application.yml

```
spring:
  datasource:
    url: "jdbc:mysql://localhost:3306/springbootdata?\
    characterEncoding=utf-8&serverTimezone=Asia/Shanghai"
    username: root
    password: root
  jpa:
    show-sql: true
```

在上述信息中，第 2～6 行代码配置了数据库的连接信息；第 7～8 行代码指定使用 Spring Data JPA 操作数据库时，将对应的 SQL 输出到控制台。

4. 创建实体类

在项目的 java 目录下创建包 com.itheima.chapter06.entity，并在该包下创建实体类 Book，具体如文件 6-3 所示。

文件 6-3 Book.java

```
import javax.persistence.*;
@Entity
@Table(name="book")
public class Book {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;       //图书编号
    @Column(name="name")
    private String name;      //图书名称
    private String author;    //图书作者
    private String press;     //图书出版社
    private String status;    //图书状态
     //setter/getter 方法，以及 toString()方法
}
```

5. 创建 Repository 接口

在 java 目录下创建包 com.itheima.chapter06.dao，在该包下自定义接口 BookRepository 继承 JpaRepository 接口，具体如文件 6-4 所示。

文件 6-4 BookRepository.java

```
import com.itheima.chapter06.entity.Book;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
@Repository
public interface BookRepository extends JpaRepository<Book,Integer> {
}
```

在上述代码中，第 4 行代码使用@Repository 注解标注接口，让项目启动时可以被 Spring

扫描到，并交由 Spring 管理；第 5 行代码指定当前接口继承 JpaRepository 接口，并指定操作的实体类是 Book，其主键类型是 Integer。

6. 创建 Service 接口和实现类

在项目的 java 目录下创建包 com.itheima.chapter06.service ,并在该包下创建 Service 接口和实现类，具体如文件 6-5 和文件 6-6 所示。

文件 6-5　BookService.java

```
1  import com.itheima.chapter06.entity.Book;
2  public interface BookService {
3      public Book findById(Integer id);
4      public Book updateById(Integer id,String name);
5      public void delById(Integer id);
6  }
```

文件 6-6　BookServiceImpl.java

```
1  import com.itheima.chapter06.dao.BookRepository;
2  import com.itheima.chapter06.entity.Book;
3  import org.springframework.beans.factory.annotation.Autowired;
4  import org.springframework.cache.annotation.*;
5  import org.springframework.stereotype.Service;
6  import org.springframework.transaction.annotation.Transactional;
7  @Service
8  @CacheConfig(cacheNames = "book")
9  @Transactional
10 public class BookServiceImpl implements BookService{
11     @Autowired
12     private BookRepository bookRepository;
13     @Cacheable(key = "#id")
14     public Book findById(Integer id){
15         //根据 id 查询图书信息
16         return bookRepository.findById(id).get();
17     }
18     @CachePut(key = "#id")
19     public Book updateById(Integer id,String name){
20         Book book=this.findById(id);
21         book.setName(name);
22         //更新图书信息
23         return bookRepository.save(book);
24     }
25     @CacheEvict(key = "#id")
26     public void delById(Integer id){
27         //根据 id 删除图书信息
28         bookRepository.deleteById(id);
29     }
30 }
```

在上述代码中 ,第 8 行代码使用@CacheConfig 注解声明当前类中共享的属性 cacheNames 的值为 book。第 13～29 行代码定义了 3 个方法，分别用于查询、更新、删除图书信息，其中，第 14～17 行代码的 findById()方法使用@Cacheable 注解进行标注，当该方法执行后会将查询到的图书信息存入缓存中 ;第 19～24 行代码的 updateById()方法使用@CachePut 注解

进行标注，当执行该方法后会更新 book 缓存中对应的图书信息；第 26～29 行代码的 delById() 方法使用@CacheEvict 注解进行标注，当执行该方法后会删除 book 缓存中对应的图书信息。

7．创建控制器类

在项目的 java 目录下创建包 com.itheima.chapter06.controller，并在该包下创建控制器类 BookController，在该类中定义查询、更新、删除图书信息的方法，具体如文件 6-7 所示。

文件 6-7 BookController.java

```
1  import com.itheima.chapter06.entity.Book;
2  import com.itheima.chapter06.service.BookService;
3  import org.springframework.beans.factory.annotation.Autowired;
4  import org.springframework.web.bind.annotation.PathVariable;
5  import org.springframework.web.bind.annotation.RequestMapping;
6  import org.springframework.web.bind.annotation.RestController;
7  @RestController
8  @RequestMapping("book")
9  public class BookController {
10     @Autowired
11     private BookService bookService;
12     @RequestMapping("/findById/{id}")
13     public Book findById(@PathVariable Integer id){
14         //根据 id 查询图书信息
15         return  bookService.findById(id);
16     }
17     @RequestMapping("/editById/{id}/{name}")
18     public Book editById(@PathVariable Integer id,@PathVariable String name){
19         //根据 id 修改图书的名称
20         return  bookService.updateById(id,name);
21     }
22     @RequestMapping("/delById/{id}")
23     public void delById(@PathVariable Integer id){
24         //根据 id 删除图书信息
25         bookService.delById(id);
26     }
27 }
```

8．创建项目启动类

在项目的 java 目录下的 com.itheima.chapter06 包下创建 Spring Boot 项目的启动类，在启动类上开启缓存，具体如文件 6-8 所示。

文件 6-8 Chapter06Application.java

```
1  import org.springframework.boot.SpringApplication;
2  import org.springframework.boot.autoconfigure.SpringBootApplication;
3  import org.springframework.cache.annotation.EnableCaching;
4  @SpringBootApplication
5  @EnableCaching
6  public class Chapter06Application {
7      public static void main(String[] args) {
8          SpringApplication.run(Chapter06Application.class, args);
9      }
10 }
```

在上述代码中，第 5 行代码使用@EnableCaching 注解开启缓存。

9. 测试缓存效果

运行文件 6-8 后，在浏览器中访问 http://localhost:8080/book/findById/3，查询图书信息，控制台输出信息和浏览器中查询到的图书信息如图 6-1 和图 6-2 所示。

从图 6-1 中可以看到，控制台输出了一条查询语句，说明查询图书信息时发送对应的 SQL 到数据库进行查询。

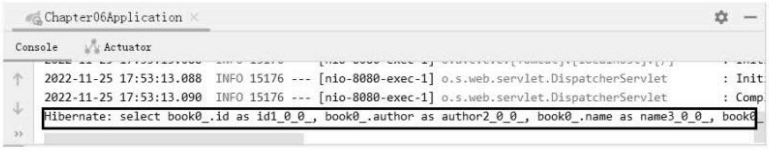

图6-1　查询图书控制台输出信息（1）

图6-2　浏览器中查询到的图书信息（1）

从图 6-2 中可以看出，浏览器中展示了 id 为 3 的图书信息，说明图书查询成功。

再次在浏览器中访问 http://localhost:8080/book/findById/3，查询 id 为 3 的图书信息，控制台输出信息如图 6-3 所示。

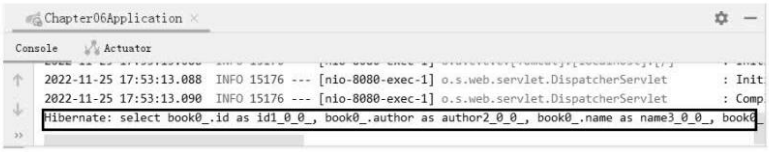

图6-3　查询图书控制台输出信息（2）

从图 6-3 可以看出，控制台没有输出新的 SQL 信息，说明此时查询到的图书信息是从缓存中获取的，而没有查询数据库。

在浏览器中访问 http://localhost:8080/book/editById/3/西游释厄传，将 id 为 3 的图书名称更新为“西游释厄传”。此时控制台输出信息和浏览器中查询到的图书信息如图 6-4 和图 6-5 所示。

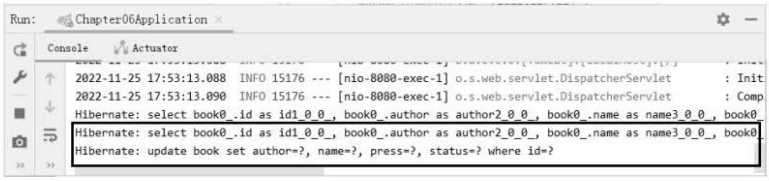

图6-4　更新图书控制台输出信息（1）

从图 6-4 可以看出，控制台输出了两条语句，即 1 条查询语句和 1 条更新语句，说明更新图书信息时，执行 BookServiceImpl 中的 updateById()方法时，发送查询和更新图书信息的 SQL 语句。

从图 6-5 可以看出，id 为 3 的图书名称更新为“西游释厄传”。

图6-5　浏览器中查询到的图书信息（2）

此时，在浏览器中再次访问 http://localhost:8080/book/findById/3，查询 id 为 3 的图书信息，控制台输出信息和浏览器中查询到的图书信息如图 6-6 和图 6-7 所示。

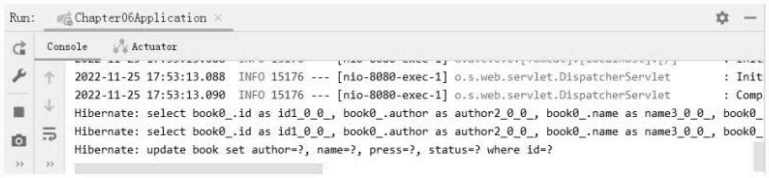

图6-6　查询图书控制台输出信息（3）

图6-7　浏览器中查询到的图书信息（3）

从图 6-6 可以看出，控制台没有输出新的 SQL 信息，但图 6-7 中展示的图书信息是更新后的图书信息，说明图书更新后将更新后的图书信息更新在缓存中，再次查询对应 id 的图书信息时，也是从缓存中查询到的。

在浏览器中访问 http://localhost:8080/book/delById/3，删除 id 为 3 的图书信息，控制台输出信息如图 6-8 所示。

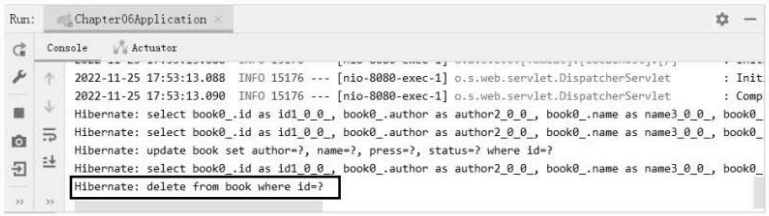

图6-8　删除图书控制台输出信息

从图 6-8 可以看出，控制台输出了一条删除的 SQL 语句，说明删除图书信息时，发送 SQL 语句到数据库中删除图书信息。

在浏览器中再次访问 http://localhost:8080/book/findById/3，查询 id 为 3 的图书信息，控制台输出信息如图 6-9 所示。

从图 6-9 可以看出，控制台输出了 1 条查询语句后输出了异常信息“ java.util.NoSuch ElementException Great breakpoint: No value present ”，说明删除图书信息时，将缓存中对应的数据也删除了，再次查询 id 为 3 的图书信息时，缓存中并不存在对应的图书信息，程序接着发送 SQL 语句到数据库进行查询，但数据库中已经不存在对应的图书信息，故抛出查询失败的异常信息。

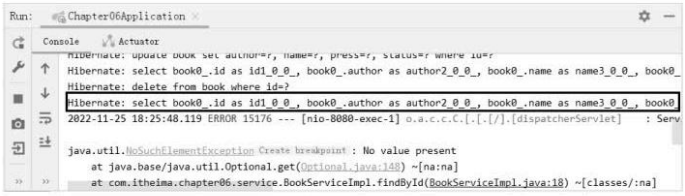

图6-9　查询图书控制台输出信息（4）

6.2　Spring Boot 整合 Ehcache 缓存

Spring 提供的缓存是一种抽象的服务，开发人员只需引入缓存接口的具体实现，而不必编写缓存的具体逻辑，便于进行缓存技术的开发与管理。在 Spring Boot 项目中，只需要引入对应缓存实现的依赖，即可使用该缓存，其中 Ehcache 是当前最快的 Java 缓存之一，下面对 Spring Boot 整合 Ehcache 缓存的相关内容进行讲解。

6.2.1　Ehcache 概述

Ehcache 是一种开源的缓存框架，它配置简单、结构清晰、功能强大，是当前使用最广泛的基于 Java 语言的缓存之一。Ehcache 可以很便捷地与其他流行的库和框架进行集成，其中，Hibernate 默认的缓存提供者就是 Ehcache。

Ehcache 能够成为目前使用最广泛的缓存框架之一，主要得益于它的以下几个特点。

1. 快速轻量

使用 Ehcache 不需要什么特别的配置，Ehcache 提供了合理的默认值不需要初始配置，并且其 API 非常简单，易于使用，可以在几分钟内启动和运行。Ehcache 的依赖关系非常简单，其核心依赖只有一个，即 SLF4J。

2. 伸缩性

Ehcache 可扩展到数百个缓存 ,并且使用堆外存储。Ehcache 可以对高并发和大型多 CPU 服务器进行优化，可以在具有几十个 CPU 内核的系统上进行高并发访问。通过添加 Terracotta，Ehcache 可以扩展到任何用例，并且在分布式部署架构上支持集群缓存。

3. 灵活性

Ehcache 提供多种缓存策略，具体如下。

（1）过期策略：Ehcache 支持基于 Cache 和基于 Element 的过期策略，每个 Cache 的存活时间都是可以设置和控制的。

（2）淘汰策略：Ehcache 提供了 LRU（Least Recently Used，最近最少使用）、LFU（Least Frequently Used，最不经常使用）和 FIFO（First Input First Output，先进先出）缓存淘汰算法。

（3）存储策略：Ehcache 提供内存和磁盘存储，Ehcache 与大多数缓存解决方案一样，提供高性能的内存和磁盘存储。

（4）动态缓存配置策略：Ehcache 可以在运行时对缓存的存活时间、空闲时间、内存和磁盘存放缓存的最大数目进行修改。

4. 标准支持

JSR107 规范是 Java 平台上的缓存规范，它定义了用于创建缓存的 API，这些 API 可以与任何缓存一起使用。Ehcache 是 JSR107 规范的完整实现，可以满足各种应用场景下的缓

存需求。

5. 可扩展性

Ehcache 插件化了与缓存相关的监听器接口、装饰器接口、缓存加载器、缓存写入器，以及异常处理程序，程序可以根据需要进行相应的拓展。

6. 监听器

Ehcache 提供了缓存管理器监听器，可以通过 CacheManagerEventListener 接口和 CacheEventListener 接口分别注册缓存管理器监听器和缓存事件监听器，提供了灵活的缓存处理机制。

7. Java 企业级应用缓存

Ehcache 为 Java 企业级应用的常见缓存场景和模式提供高质量实现。例如，Ehcache 对数据库查询缓存、对象缓存、分布式缓存提供了支持。另外，Ehcache 是 Hibernate 默认的二级缓存。

8. 开源协议 Apache License 2.0

Ehcache 采用更新后的 Apache License 2.0 发布。Apache License 是一种友好的许可证，其可使在其他开源项目或商业产品中包含 Ehcache 变得安全且容易。

Ehcache 支持分层缓存，所有分层缓存都可以单独使用，Ehcache 支持的分层选项如下。

- 堆：堆内存速度快，不需要序列化，但是容量有限。
- OffHeap（堆外）：OffHeap 存储只在企业版本的 Ehcache 中提供，原理是利用 NIO 的 DirectByteBuffers 实现，比存储到磁盘上快，而且完全不受 GC（Garbage Collection，垃圾收集）的影响，可以保证响应时间的稳定性；OffHeap 存储的对象必须在存储过程中进行序列化，读取则进行反序列化操作，它的速度大约比堆内存储慢一个数量级。
- 磁盘：“磁盘”层数据存储在硬盘上，磁盘越快、越专用，访问数据的速度就越快。存储在磁盘上的数据进行写入和读取时必须进行对应的序列化或反序列化，因此比堆存储和 OffHeap 存储要慢。
- 集群：使用集群层存储意味着客户端连接到 Terracotta 服务器阵列时，缓存的数据存储在 Terracotta 服务器，这也是 JVM 之间共享缓存的一种方式。

如果需要使用多个层，必须遵守以下要求。

（1）在多层设置中必须始终存在堆层。

（2）不能将磁盘层和集群层组合在一起。

（3）层的大小应该按照金字塔的方式调整，即金字塔上更高的层比更低的层使用更少的内存。

Ehcache 各层之间相互关联，通常堆内存比计算机的总内存受限制更多，堆外内存比磁盘或集群上可用的内存受限制更多。这就形成了多层建筑的典型金字塔形状，如图 6-10 所示。

图6-10 Ehcache分层架构

从图 6-10 可以看出，Ehcache 要求磁盘层的大小大于堆外层的大小，堆外层的大小大

于堆层的大小。

6.2.2　整合 Ehcache

通过前面的学习，读者对 Ehcache 有了基本的了解。下面将 Spring Boot 和 Ehcache 进行整合，进一步演示在 Spring Boot 项目中使用 Ehcache，具体如下。

1. 配置依赖

在项目 chapter06 的 pom.xml 文件中配置添加 Ehcache 的依赖，具体代码如下。

```
<dependency>
   <groupId>net.sf.ehcache</groupId>
   <artifactId>ehcache</artifactId>
</dependency>
```

在上述代码中，导入的是 Ehcache 框架的依赖，并没有导入 Ehcache 启动器之类的依赖，是因为 6.1.3 小节中导入的 spring-boot-starter-cache 启动器中包含了 Spring Boot 缓存技术整合的通用方式，不管整合哪种具体的缓存技术，操作方式一样。此处只需给出具体实现的缓存技术即可，这也体现出 Spring Boot 统一同类技术的整合方式的优势。

2. 设置配置信息

在项目 chapter06 的 application.yml 文件中添加 Ehcache 的相关配置，具体代码如下。

```
 cache:
   type: ehcache
   ehcache:
     config: ehcache.xml
```

在上述配置中，type 属性用于指定配置缓存的类型为 ehcache，这个类型不能是自定义的，是因为 Spring Boot 可以整合的缓存技术中包含有 ehcach；config 属性用于指定 Ehcache 的配置文件，Ehcache 的配置有独立的配置文件格式，通过 config 指定可以便于读取相应配置。

在项目 chapter06 的 resources 目录下创建文件 ehcache.xml，在该文件中添加 Ehcache 的相关配置，具体如文件 6-9 所示。

文件 6-9　ehcache.xml

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <ehcache xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3          xsi:noNamespaceSchemaLocation="http://ehcache.org/ehcache.xsd"
4          updateCheck="false">
5      <diskStore path="D:\ehcache" />
6      <cache
7             name="book"
8             eternal="true"
9             diskPersistent="true"
10            maxElementsInMemory="1000"
11            overflowToDisk="false"
12            memoryStoreEvictionPolicy="LRU">
13        <bootstrapCacheLoaderFactory
14    class="net.sf.ehcache.store.DiskStoreBootstrapCacheLoaderFactory"
15              properties="bootstrapAsynchronously=true">
16        </bootstrapCacheLoaderFactory>
17    </cache>
18 </ehcache>
```

在上述代码中，第 5 行代码用于指定缓存数据保存到本地磁盘的文件路径；第 6～17 行代码用于指定缓存的配置信息，其中各属性及标签的作用为：

第 7 行代码 name 用于指定缓存的名称，对应@Cacheable 等缓存注解中 cacheNames 和 value 的值；

第 8 行代码 eternal 用于指定是否永久存在，设置为 true 时不会被清除，但会与 timeout 属性冲突；

第 9 行代码 diskPersistent 用于指定是否启用磁盘持久化；

第 10 行代码 maxElementsInMemory 用于指定最大缓存数量；

第 11 行代码 overflowToDisk 用于指定超过最大缓存数量是否持久化到磁盘；

第 12 行代码 memoryStoreEvictionPolicy 用于指定缓存清除策略；

第 13～16 行代码用于指定缓存启动时，从硬盘加载数据进行缓存的初始化。

<cache>标签中除了上述常用属性外，还有以下属性也经常在项目中使用，示例如下。

```
timeToIdleSeconds="10"
timeToLiveSeconds="10"
```

在上述代码中，timeToIdleSeconds 用于指定失效前的空闲秒数，当 eternal 为 false 时这个属性才有效，设置过长缓存容易溢出，设置过短则无效果，一般用于记录时效性数据，例如验证码 ,timeToLiveSeconds 用于指定最大存活时间 ,也是当 eternal 为 false 时这个属性才有效。

3. 设置实体类

由于本案例要将缓存序列化到本地磁盘，所以业务中操作的实体类需要实现序列化接口。使文件 6-3 中的 Book 类实现序列化接口，修改后如文件 6-10 所示。

文件 6-10 Book.java

```
1  import javax.persistence.*;
2  @Entity
3  @Table(name="book")
4  public class Book implements  Serializable{
5      @Id
6      @GeneratedValue(strategy = GenerationType.IDENTITY)
7      private Integer id;       //图书编号
8      @Column(name="name")
9      private String name;      //图书名称
10     private String author;    //图书作者
11     private String press;     //图书出版社
12     private String status;    //图书状态
13      //……setter/getter 方法，以及 toString()方法
14 }
```

在上述代码中，只添加了实现 Serializable 接口的代码，其他代码与文件 6-3 一致。

4. 测试缓存效果

运行文件 6-8 后，在浏览器中访问 http://localhost:8080/book/findById/5，查询图书信息，控制台输出信息和浏览器中查询到的图书信息如图 6-11 和图 6-12 所示。

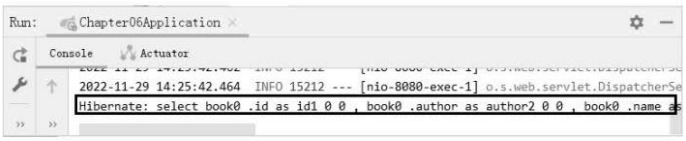

图6-11 查询图书控制台输出信息（5）

图6-12　浏览器中查询到的图书信息（4）

从图 6-11 中可以看出，控制台输出了一条查询语句，说明查询图书信息时发送对应的 SQL 到数据库进行查询。

从图 6-12 中可以看出，浏览器中展示了 id 为 5 的图书信息，说明图书查询成功。

再次在浏览器中访问 http://localhost:8080/book/findById/5，查询 id 为 5 的图书信息，控制台输出信息如图 6-13 所示。

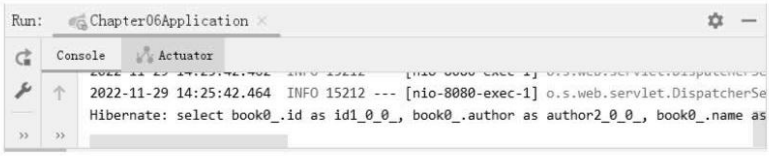

图6-13　查询图书控制台输出信息（6）

从图 6-13 可以看出，控制台没有输出新的 SQL 信息，说明此时查询到的图书信息是从缓存中获取到的，而没有查询数据库。

在浏览器中访问 http://localhost:8080/book/editById/5/观堂别集，将 id 为 5 的图书名称更新为“ 观堂别集 ”。此时控制台输出信息和浏览器中查询到的图书信息如图 6-14 和图 6-15 所示。

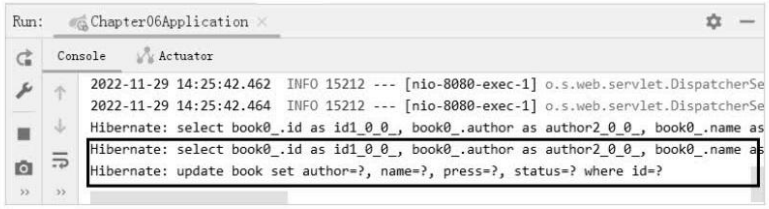

图6-14　更新图书控制台输出信息（2）

图6-15　浏览器中查询到的图书信息（5）

从图 6-14 可以看出，控制台输出了两条语句，即 1 条查询语句和 1 条更新语句，说明更新图书信息时，执行 BookServiceImpl 中的 updateById()方法时，发送查询和更新图书信息的 SQL 语句。

从图 6-15 可以看出，id 为 5 的图书名称更新为“ 观堂别集 ”。

此时，单击 IDEA 控制台左侧的按钮，正常关闭程序，如图 6-16 所示。

程序关闭后，打开之前设置的缓存数据保存到本地磁盘的文件路径，如图 6-17 所示。

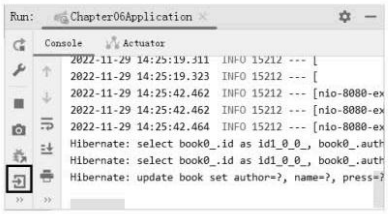

图6-16 正常关闭程序

图6-17 保存缓存数据到本地磁盘的文件路径

从图 6-17 可以看出，保存缓存数据到本地磁盘的文件路径下自动生成了后缀名为.data 的数据文件，以及后缀名为.index 的索引文件，这两个文件都是 Ehcache 自动序列化到本地的缓存文件。

此时，在此运行文件 6-8 后，在浏览器中访问 http://localhost:8080/book/findById/5，查询图书信息，控制台输出信息和浏览器中查询到的图书信息如图 6-18 和图 6-19 所示。

从图 6-18 可以看出，控制台并没有输出任何 SQL 语句。

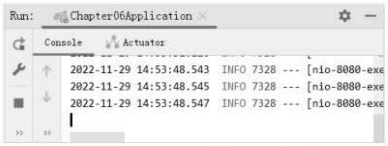

图6-18 查询图书控制台输出信息（7）

图6-19 浏览器中查询到的图书信息（6）

从图 6-19 可以看出，浏览器中展示了 id 为 5 的图书信息，说明图书查询成功，程序启动后，将之前序列化在磁盘中的缓存添加到内存的缓存中。

需要注意的是，为了确保程序关闭时成功将内存中的缓存数据序列化到本地磁盘，建议采用正常关闭程序的方式，直接杀死进程等非正常关闭方式可能导致缓存序列化到本地磁盘失败。

6.3 Spring Boot 整合 Redis 缓存

Redis 是一款在互联网领域被广泛应用的高性能缓存数据库，在使用 Redis 的过程中，需要具备科学的信息管理思想，注重缓存的安全性、可执行性和完备性，建立完整、严格的审计机制，确保程序正常运行并避免缓存异常流动。

Redis 优秀的数据处理能力和丰富的数据结构，使其可以使用的业务场景非常广泛，既可以作为数据库使用，也可以作为缓存使用。关于 Redis 作为数据库使用的相关内容在第 5 章已经讲解了，下面通过案例演示 Redis 作为缓存在 Spring Boot 项目中的使用，具体如下。

1. 配置依赖

本案例只演示 Redis 作为缓存在 Spring Boot 项目中的使用，为了能展示缓存效果，本案例演示时候将 Ehcache 的相关配置进行注释。在项目 chapter06 的 pom.xml 文件中将 Ehcache 的依赖进行注释，配置添加 Redis 的依赖，具体代码如下。

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-redis</artifactId>
```

```
</dependency>
```

2. 设置配置信息

在项目 chapter06 的 application.yml 文件中将原有配置的 Ehcache 相关信息进行注释，添加 Redis 缓存相关的配置，具体代码如下。

```
spring:
  redis:
    host: localhost
    port: 6379
  cache:
    type: redis
    redis:
      use-key-prefix: true
      key-prefix: book_
      cache-null-values: false
      time-to-live: 400s
```

本案例不是对原始的 Redis 进行配置，而是需要将 Redis 作为缓存进行配置，需要注意的是，Redis 作为缓存使用的相关配置隶属于 spring.cache.redis 节点。上述配置中，use-key-prefix 用于指定是否开启 key 前缀；key-prefix 用于指定 key 前缀；cache-null-values 用于指定是否缓存空值；time-to-live 用于指定缓存存活时间。

3. 测试缓存效果

启动 Redis 服务，运行文件 6-8 后，在浏览器中访问 http://localhost:8080/book/findById/5，查询图书信息，控制台输出信息和浏览器中查询到的图书信息如图 6-20 和图 6-21 所示。

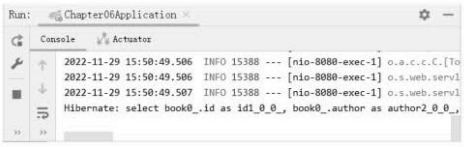

图6-20　查询图书控制台输出信息（8）

图6-21　浏览器中查询到的图书信息（7）

从图 6-20 可以看出，控制台输出了一条查询语句，说明查询图书信息时发送对应的 SQL 到数据库进行查询。

从图 6-21 可以看出，浏览器中展示了 id 为 5 的图书信息，说明图书查询成功。

再次查询 id 为 5 的图书信息，控制台输出信息如图 6-22 所示。

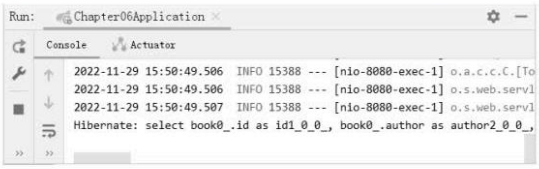

图6-22　浏览器中查询到的图书信息（8）

从图 6-22 可以看出，控制台没有输出新的 SQL 信息，说明此时查询到的图书信息是从缓存中获取的，而没有查询数据库。

打开 RESP.app，Redis 中的数据如图 6-23 所示。

从图 6-23 可以看出，Redis 中存放了一个“book_”开头的键，该键对应的数据类型为 String 类型。由于 Redis 默认的序列化机制为 JDK 的序列化策略，所以存储的内容展示形式

为二进制。

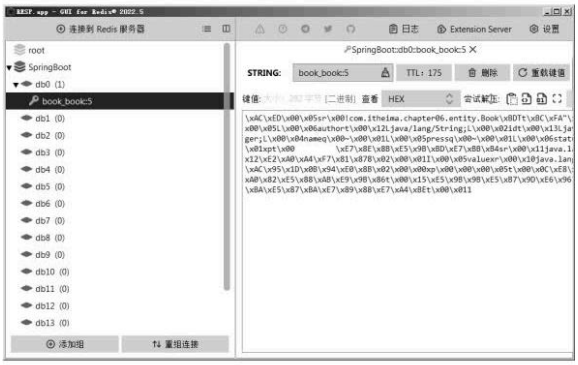

图6-23 Redis中的数据

通过本章使用 Spring Boot 整合不同的缓存技术可以发现，切换缓存技术时，原始代码不需要有任何修改，只需配置一组缓存配置信息就可以变更缓存供应商，这也是 Spring Boot 提供了统一的缓存操作接口的优势，这使得变更缓存实现并不影响原始代码的书写。

6.4 本章小结

本章主要对 Spring Boot 整合缓存进行了讲解。首先讲解了 Spring Boot 默认缓存管理；然后讲解了 Spring Boot 整合 Ehcache 缓存；最后讲解了 Spring Boot 整合 Redis 缓存。通过本章的学习，希望读者可以在 Spring Boot 项目中正确应用缓存技术，为后续更深入地学习 Spring Boot 做好铺垫。

6.5 本章习题

一、填空题

1．Spring Boot 项目中开启缓存管理后，如果没有任何缓存组件，默认使用_____缓存组件进行管理。

2．当配置类上使用_____注解，会默认提供实现 CacheManager 的缓存组件，并通过 AOP 将缓存行为添加到应用程序。

3．_____注解主要用于统筹管理类中所有声明式缓存注解的公共属性。

4．@Cacheable 注解的_____属性和 cacheNames 属性都可以指定缓存的名称。

5．Spring Boot 项目中，全局配置文件中 spring.cache.redis.key-prefix 用于指定 Redis 中缓存 key 的_____。

二、判断题

1．CacheManager 是缓存管理器，基于缓存名称对缓存进行管理。（　　）

2．标注@Cacheable 注解的方法执行时，会先查询缓存，如果查询到缓存数据，则不执行该方法。（　　）

3．@Cacheable 注解的 value 和 cacheNames 属性作用相同，且要二选一使用。（　　）

4．Ehcache 配置文件中，只有 eternal 为 false 时 timeToLiveSeconds 的设置才有效。（　　）

5 .Ehcache 要将缓存持久化到本地磁盘 ,则操作的实体类需要实现序列化接口。(　　)

三、选择题

1．下列选项中，关于 Spring Boot 默认缓存方案说法错误的是（　　）。

A．Spring Boot 继承了 Spring 框架的缓存管理功能。

B．Spring 的缓存机制将提供的缓存作用于 Java 方法上。

C．Spring 的默认缓存方案通过 Cache 和 CacheManager 接口统一不同的缓存技术。

D．Spring 中的 Cache 接口制定了管理 Cache 的规则。

2．下列选项中，关于@Cacheable 注解的属性说明错误的是（　　）。

A．value 用于指定 CacheManager 的实现。

B．key 用于指定缓存数据的 key。

C．keyGenerator 用于指定缓存数据的 key 的生成器。

D．cacheManager 用于指定缓存管理器。

3．下列选项中，关于声明式缓存注解的描述错误的是（　　）。

A．@EnableCaching 是 Spring 框架提供的用于开启基于注解的缓存支持的注解。

B．@CachePut 注解的执行顺序是先将方法结果更新到缓存中，再进行方法调用。

C．@CachePut 注解的作用是更新缓存数据。

D．@CacheEvict 注解的默认执行顺序是先进行方法调用，然后将缓存清除。

4．下列选项中，关于类上标注@CacheConfig(cacheNames = "book")代码的描述正确的是（　　）。

A．声明当前类中共享的属性 cacheNames 的值为 book。

B．类中所有方法上使用缓存注解时 cacheNames 属性的值默认都为 book。

C．如果类的方法上再次定义 cacheNames 属性，那么 cacheNames 属性的值最终为方法上指定的属性值。

D．类的方法上可以再次定义 cacheNames 属性的值。

5．下列选项中，Ehcache 配置文件中用于指定是否启用磁盘持久化的属性是（　　）。

A．external　　B．diskPersistent

C．maxElementsInMemory　　D．memoryStoreEvictionPolicy

第 7 章

Spring Boot安全管理

学习目标

★ 了解安全框架概述，能够简述 Spring Security 和 Shiro 的作用

★ 掌握 Spring Security 入门案例，能够基于 Spring Boot 项目完成 Spring Security 入门案例

★ 了解 Spring Security 结构总览，能够简述 Spring Security 过滤器处理请求的流程

★ 熟悉 Spring Security 认证流程，能够简述 Spring Security 的认证流程

★ 掌握 Spring Security 自定义身份认证，能够基于内存身份认证、JDBC 身份认证和自定义 UserDetailsService 实现用户身份认证

★ 熟悉 Spring Security 授权流程，能够简述 Spring Security 的授权流程

★ 掌握 Spring Security 自定义授权，能够使用 Web 授权和方法授权实现用户授权管理

★ 掌握动态展示菜单，能够通过 Spring Security 的授权管理实现动态展示菜单

★ 掌握 Spring Security 会话管理，能够在 Spring Security 中获取认证后的用户信息，以及进行会话控制

★ 掌握 Spring Security 用户退出，能够在 Spring Boot 项目中使用 Spring Security 实现用户退出

实际开发中，开发者为了确保 Web 应用的安全性，通常需要保护 Web 应用的用户信息、数据信息等资源不受侵害，例如，对于某些指定的功能，需要先对访问的用户进行身份验证，验证通过后还需要具备相关权限之后才可以操作。下面将对 Spring Boot 的安全管理进行详细讲解。

7.1 安全框架概述

Java 中的安全框架通常是指解决 Web 应用安全问题的框架，如果开发 Web 应用时没有使用安全框架，开发者需要自行编写代码增加 Web 应用安全性。自行实现 Web 应用的安全性并不容易，需要考虑不同的认证和授权机制、网络关键数据传输加密等多方面的问题，为此 Web 应用中通常会选择使用一些成熟的安全框架，这些安全框架基本都提供了一套 Web 应用安全性的完整解决方案，以便提升 Web 应用的安全性。

Java 中常用的安全框架有 Spring Security 和 Shiro，这两个安全框架都提供了强大功能，

可以很容易实现 Web 应用的很多安全防护。下面对这两个安全框架的特点进行讲解。

1. Spring Security

Spring Security 是 Spring 生态系统中重要的一员，是一个基于 AOP 思想和 Servlet 过滤器实现的安全框架，它提供了完善的认证机制和方法级的授权功能，是一款非常优秀的权限管理框架。Spring Security 伴随着 Spring 生态系统不断修正、升级，使用 Spring Security 减少了为企业系统安全控制编写大量重复代码的工作，在 Spring Boot 项目中使用 Spring Security 十分简单。

使用 Spring Security 可以很方便地实现 Authentication（认证）和 Authorization（授权），其中认证是指验证用户身份的过程，授权是指验证用户是否有权限访问特定资源的过程。Spring Security 在架构上将认证与授权分离，并提供了扩展点。

Spring Security 具有以下的特点。

- 灵活：Spring Security 提供了一系列可扩展的模块，可以根据具体需求进行选择和配置。
- 安全：Spring Security 集成了一系列安全措施，包括 XSS（Cross-Site Scripting，跨站脚本）攻击防范、CSRF（Cross-Site Request Forgery，跨站请求伪造）攻击防范、点击劫持攻击防范等。
- 易用：Spring Security 提供了一系列快捷配置选项，可以使开发人员轻松地实现认证和授权等功能。
- 社区支持：Spring Security 作为 Spring 生态系统的一部分，与 Spring 无缝整合，得到了社区广泛的支持和更新维护。

2. Shiro

Shiro 是 Apache 旗下一个开源框架，它将软件系统的安全认证相关功能抽取出来，实现用户身份认证、授权、加密、会话管理等功能，组成了一个通用的安全认证框架。

Shiro 具有以下特点。

- 易于理解的 Java Security API。
- 简单的身份认证，支持 LDAP（Lightweight Directory Access Protocol，轻量目录访问协议），JDBC 等多种数据源。
- 支持对角色的简单鉴权，也支持细粒度的鉴权。
- 支持一级缓存，以提升应用程序的性能。
- 内置的基于 POJO（Plain Ordinary Java Object）企业会话管理，适用于 Web 和非 Web 的环境。
- 不与任何的框架或者容器捆绑，可以独立运行。

从上述内容可知，不管是 Spring Security 还是 Shiro，在进行安全管理的过程中都涉及权限管理的两个重要概念：认证和授权。权限管理是指根据系统设置的安全规则或者安全策略，用户可以访问且只能访问自己被授权的资源。

实现权限管理通常需要三个对象，分别为用户、角色、权限，这三个对象的说明如下。

- 用户：主要包含用户名、密码和当前用户的角色信息，可以实现认证操作。
- 角色：主要包含角色名称、角色描述和当前角色拥有的权限信息，可以实现授权操作。
- 权限：权限也可以称为菜单，主要包含当前权限名称、URL 地址等信息，可以实

现动态展示菜单。

Spring Security 和 Shiro 两者都有各自的优点，开发者可以根据实际项目需求进行选择。Spring Security 作为 Spring 家族的一员，Spring Boot 为 Spring Security 提供了自动化配置方案，仅需要少量的配置即可使用 Spring Security，通常在 Spring Boot 项目中使用 Spring Security 解决安全问题更为方便。

7.2 Spring Security 基础入门

了解了 Spring Security 的特点之后，下面通过 Spring Security 入门案例和结构总览对 Spring Security 的基础知识进行讲解。

7.2.1 Spring Security 入门案例

本入门案例主要演示 Spring Security 在 Spring Boot 中的安全管理效果。为了更好地使用 Spring Boot 整合实现 Spring Security 安全管理功能，体现案例中 Authentication（认证）和 Authorization（授权）功能的实现，本案例在 Spring Boot 项目中结合 Spring MVC 和 Thymeleaf 实现访问图书管理后台页面，具体如下。

1. 创建 Spring Boot 项目

使用 Spring Initializr 方式创建一个名为 chapter07 的 Spring Boot 项目，在 Dependencies 依赖选择中选择 Spring Web、Thymeleaf 和 Spring Security 的依赖，然后根据提示完成项目创建。其中，创建项目时的选择依赖如图 7-1 所示。

2. 导入页面资源

在项目的 resources 目录的 templates 和 static 文件夹中，分别引入案例的页面文件，以及页面所需的静态资源文件，引入资源文件后项目的文件结构如图 7-2 所示。

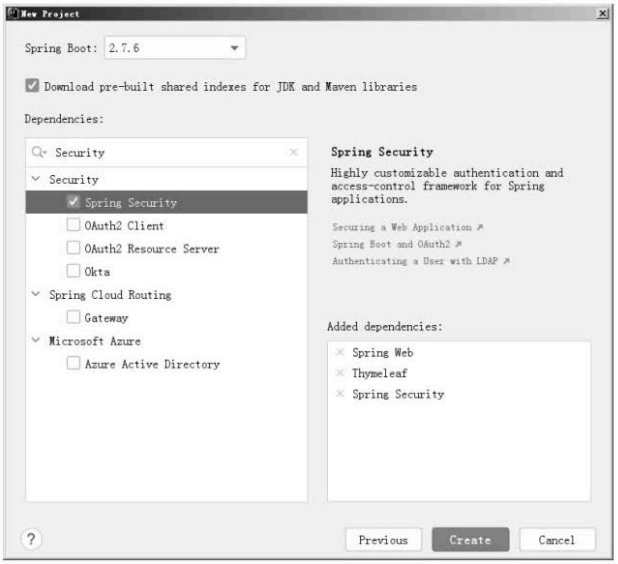

图7-1 Spring Boot项目的选择依赖

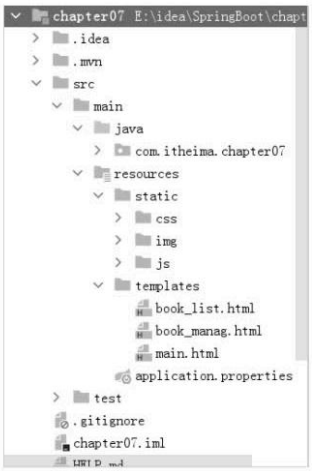

图7-2 项目的文件结构

图 7-2 中导入的页面文件有 3 个，其中 main.html 为图书管理的主页面；book_list.html

为图书列表页面，该页面展示所有可阅读的图书信息；book_manag.html 为图书管理页面，该页面可以对图书进行增删改查等操作。这 3 个页面的核心代码如文件 7-1～文件 7-3 所示。

文件 7-1　main.html

```
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<html>
<head>
    <!--……静态资源导入代码，以及页面头部代码-->
   <!-- 导航侧栏 -->
   <aside class="main-sidebar">
      <section class="sidebar">
         <ul class="sidebar-menu">
            <li >
               <a th:href="@{/book/list}" target="iframe">
                  <i class="fa fa-circle-o"></i>图书阅读
               </a>
            </li>
            <li >
               <a th:href="@{/book/admin/manag}" target="iframe">
                  <i class="fa fa-circle-o"></i>图书管理
               </a>
            </li>
         </ul>
      </section>
      <!-- /.sidebar -->
   </aside>
   <!-- 导航侧栏 /-->
   <!-- 内容展示区域 -->
   <div class="content-wrapper">
      <iframe         width="100%"         id="iframe"         name="iframe"
onload="SetIFrameHeight()"
            frameborder="0" "></iframe>
   </div>
</div>
</body>
</html>
```

在上述代码中，第 8～19 行代码在页面左侧创建两个超链接标签，单击不同的标签可发送不同的请求；第 25～28 行代码用于指定内容展示区域。

文件 7-2　book_list.html

```
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
   <meta charset="utf-8">
   <title>图书阅读</title>
   <link rel="stylesheet" th>
   <link rel="stylesheet" th:href="@{/css/bootstrap.css}">
   <link rel="stylesheet" th:href="@{/css/AdminLTE.css}">
</head>
<body>
<div class="box-body">
   <div class="table-box">
```

```
12        <!-- 数据表格 -->
13        <table id="dataList" class="table table-bordered  text-center">
14            <thead>
15            <tr>
16                <th>图书名称</th>
17                <th>图书作者</th>
18                <th>出版社</th>
19                <th>操作</th>
20            </tr>
21            </thead>
22            <tbody>
23                <tr>
24                    <td >Spring Cloud 微服务架构开发</td>
25                    <td >黑马程序员</td>
26                    <td >人民邮电出版社</td>
27                    <td class="text-center">
28                        <button  type="button" class="btn bg-olive btn-xs">
29                            阅读
30                        </button>
31                    </td>
32                </tr>
33                <tr>
34                    <td >Spring Boot 企业级开发教程</td>
35                    <td >黑马程序员</td>
36                    <td >人民邮电出版社</td>
37                    <td class="text-center">
38                        <button  type="button" class="btn bg-olive btn-xs">
39                            阅读
40                        </button>
41                    </td>
42                </tr>
43            </tbody>
44        </table>
45    </div>
46 </div>
47 </body>
48 </html>
```

在上述代码中，第 13～44 行代码展示两条图书信息。

文件 7-3　book_manag.html

```
1  <html lang="en" xmlns:th="http://www.thymeleaf.org">
2  <head>
3      <meta charset="utf-8">
4      <title>图书管理</title>
5      <link rel="stylesheet" th>
6      <link rel="stylesheet" th:href="@{/css/bootstrap.css}">
7      <link rel="stylesheet" th:href="@{/css/AdminLTE.css}">
8  </head>
9  <body>
10 <div class="box-body">
11     <div class="pull-left">
```

```
12        <div class="btn-group">
13            <button type="button" class="btn btn-default"> 新增
14            </button>
15        </div>
16    </div>
17    <div class="table-box">
18        <!-- 数据表格 -->
19        <table id="dataList" class="table table-bordered  text-center">
20            <thead>
21            <tr>
22                <th>图书名称</th>
23                <th>图书作者</th>
24                <th>出版社</th>
25                <th>操作</th>
26            </tr>
27            </thead>
28            <tbody>
29                <tr>
30                    <td >Spring Cloud 微服务架构开发</td>
31                    <td >黑马程序员</td>
32                    <td >人民邮电出版社</td>
33                    <td class="text-center">
34                        <button  type="button" class="btn bg-olive btn-xs">
35                            编辑
36                        </button>
37                        <button  type="button" class="btn bg-olive btn-xs">
38                            删除
39                        </button>
40                    </td>
41                </tr>
42                <tr>
43                    <td >Spring Boot 企业级开发教程</td>
44                    <td >黑马程序员</td>
45                    <td >人民邮电出版社</td>
46                    <td class="text-center">
47                        <button  type="button" class="btn bg-olive btn-xs">
48                            编辑
49                        </button>
50                        <button  type="button" class="btn bg-olive btn-xs">
51                            删除
52                        </button>
53                    </td>
54                </tr>
55            </tbody>
56        </table>
57    </div>
58 </div>
59 </body>
60 </html>
```

在上述代码中，第 13～14 行代码提供了“ 新增 ”按钮；第 19～56 行代码展示了可进

行管理的图书信息。

3. 创建控制器类

在项目的 java 目录下创建包 com.itheima.chapter07.controller，并在该包下创建实体类控制器类 BookController，在该类中定义处理图书列表和图书管理请求的方法，具体如文件 7-4 所示。

文件 7-4 BookController.java

```
1  import org.springframework.stereotype.Controller;
2  import org.springframework.ui.Model;
3  import org.springframework.web.bind.annotation.RequestMapping;
4  @Controller
5  @RequestMapping("book")
6  public class BookController {
7      @RequestMapping("list")
8      public String findList() {
9          return "book_list";
10     }
11     @RequestMapping("admin/manag")
12     public String findManagList() {
13         return "book_manag";
14     }
15 }
```

在上述代码中，第 8～14 行定义的两个方法中只是简单对请求进行跳转，并跳转到对应的页面。

4. 添加配置类

在项目的 java 目录下创建包 com.itheima.chapter07.config，并在该包下创建配置类 WebMvcConfig，在该类中添加视图路径映射，实现访问项目首页自动映射到后台管理首页，具体如文件 7-5 所示。

文件 7-5 WebMvcConfig.java

```
1  import org.springframework.context.annotation.Configuration;
2  import
org.springframework.web.servlet.config.annotation.ViewControllerRegistry;
3  import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;
4  @Configuration
5  public class WebMvcConfig implements WebMvcConfigurer {
6      @Override
7      public void addViewControllers(ViewControllerRegistry registry) {
8          registry.addViewController("/").setViewName("/main.html");
9      }
10 }
```

在上述代码中，第 8 行代码用于将路径“ / ”映射到视图“ /main.html ”。

5. 测试项目效果

启动项目，在浏览器中访问“ http://localhost:8080/ ”，浏览器效果如图 7-3 所示。

从图 7-3 可以看出，浏览器并没有展示 main.html 页面，而是展示了一个并未手动导入的页面，该页面中提供了一个用户登录表单，并且浏览器地址栏中的地址也变为“ http://localhost:8080/login ”。

出现以上情况的原因在于 Spring Boot 项目中加入 spring-boot-starter-security 依赖启动器后，Spring Boot 为 Spring Security 提供了自动化配置，Spring Security 提供了一些默认的安全管理功能，如果发送当前请求时并未进行认证，会将请求重定向到“ /login ”，如果项目没有指定“ /login ”对应的页面，则跳转到 Spring Security 自带的默认登录页面。

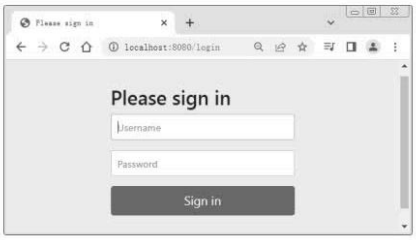

图7-3　浏览器效果

因为当前所创建的项目中并没有设置用户登录的相关信息，所以如果需要使用 Spring Security 自带的默认登录页面进行登录，需要使用 Spring Security 提供的用户登录信息。查看 IDEA 控制台信息，信息中包含一些特别的内容，如图 7-4 所示。

图7-4　控制台信息

从图 7-4 中可以看出，控制台输出的信息中提示“Using generated security password：9cca15d8-5344-4139-8406-038299f99634，This generated password is for development use only.Your security configuration must be updated before running your application in production”等消息。上述消息中的密码为 Spring Security 默认提供的用户登录密码，会在项目每次启动时随机生成，其用户名为 user。

下面在图 7-3 中使用 Spring Security 提供的账号，以及生成的随机密码进行登录，登录后的效果如图 7-5 所示。

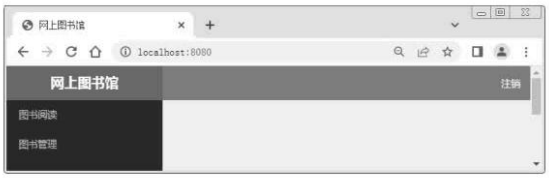

图7-5　登录后的效果

从图 7-5 可以看出，登录成功后继续访问之前发起的请求，会跳转到项目的 main.html 页面。

7.2.2　Spring Security 结构总览

Spring Security 可以对请求进行安全访问控制，安全访问控制的本质其实就是对所有进入系统的请求进行拦截，校验每个请求是否能够访问它所期望访问的资源。Spring Security 基于 Filter 实现对 Web 资源的保护。下面通过 Spring Security 中的过滤器对 Spring Security 的整体结构进行学习。

当初始化 Spring Security 时，会创建一个类型为 org.springframework.security. web.FilterChainProxy、名称为 springSecurityFilterChain 的过滤器，这个过滤器实现了 javax.servlet.Filter 接口，外部请求系统资源时会经过此过滤器，该过滤器处理请求的过程如图 7-6 所示。

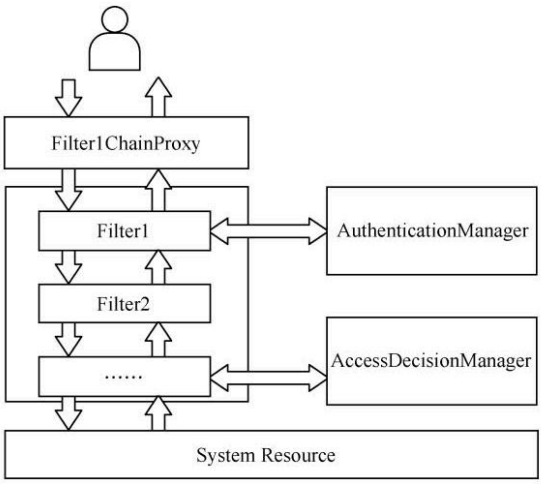

图7-6 过滤器处理请求的过程

从图 7-6 可以看出，FilterChainProxy 接收到请求后，进一步将请求传递到 Filter。其实 FilterChainProxy 是一个代理，真正起作用的是 FilterChainProxy 中 SecurityFilterChain 对象，而这个 SecurityFilterChain 对象中封装了多个 Filter，这些 Filter 作为 Bean 被 Spring 管理，它们是 Spring Security 核心，各司其职，但它们并不直接处理用户的认证，也不直接处理用户的授权，而是交给 AuthenticationManager（认证管理器）和 AccessDecisionManager（决策管理器）进行处理。

Spring Security 安全管理的实现主要是由 SecurityFilterChain 中一系列过滤器相互配合完成的，下面对 SecurityFilterChain 中主要的几个过滤器及其作用分别进行说明，具体如下。

- SecurityContextPersistenceFilter：是整个拦截过程的入口和出口，也就是第一个和最后一个拦截器。使用 SecurityContextRepository 在 Session 中保存或更新一个 SecurityContext，并将 SecurityContext 给以后的过滤器使用，为后续 Filter 建立所需的上下文。SecurityContext 中存储了当前用户的认证和权限信息。
- UsernamePasswordAuthenticationFilter：用于处理来自表单提交的认证，提交的表单必须提供对应的用户名和密码，其内部还应有登录成功或失败后进行处理的 AuthenticationSuccessHandler 和 AuthenticationFailureHandler。
- FilterSecurityInterceptor：用于保护 Web 资源，获取所配置资源访问的授权信息，根据 SecurityContextHolder 中存储的用户信息来决定其是否有权限。
- CsrfFilter：Spring Security 会对所有 Post 请求验证是否包含系统生成的 CSRF 的 Token 信息，如果不包含，则报错，起到防止 CSRF 攻击的效果。
- ExceptionTranslationFilter：能够捕获来自 FilterChain 所有的异常，并进行处理，但是它只会处理两类异常，即 AuthenticationException 和 AccessDeniedException，对应捕获到的其他异常会继续抛出。
- DefaultLoginPageGeneratingFilter：如果没有在配置文件中指定认证页面，则由该过滤器生成一个默认认证页面。

7.3 Spring Security 认证管理

认证是 Spring Security 的核心功能之一，Spring Security 所提供的认证可以更好地保护

系统的隐私数据与资源，只有当用户的身份合法后方可访问该系统的资源。Spring Security 提供了默认的认证相关配置，开发者也可以根据自己实际的环境进行自定义认证配置。下面对 Spring Security 的认证流程以及自定义身份认证进行讲解。

7.3.1　Spring Security 认证流程

用户认证就是判断一个用户的身份是否合法的过程，用户访问系统资源时系统要求验证用户的身份信息，身份合法方可继续访问，否则拒绝其访问。下面通过一张图展示 Spring Security 的认证流程，如图 7-7 所示。

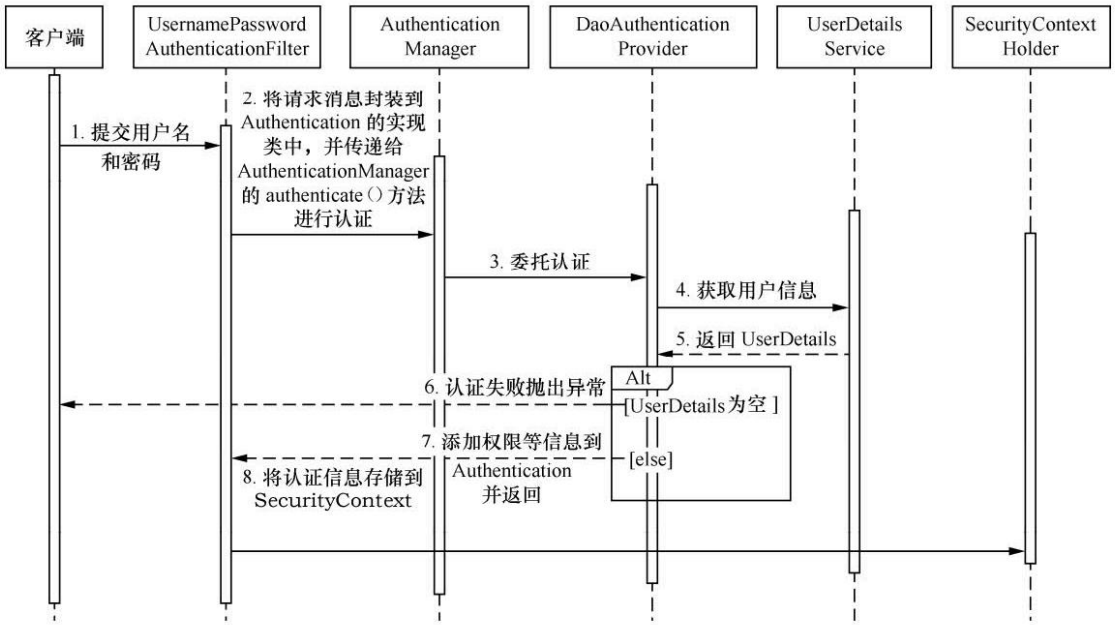

图7-7　Spring Security的认证流程

下面结合图 7-7 对 Spring Security 的认证流程进行详细介绍。

用户提交用户名和密码进行认证请求后，被 SecurityFilterChain 中的 UsernamePasswordAuthenticationFilter 过滤器获取到，将用户名和密码封装到 UsernamePasswordAuthenticationToken 对象中，该对象为 Authentication 的实现类。

过滤器将封装用户名和密码的 Authentication 对象提交至 AuthenticationManager（认证管理器）进行认证。

AuthenticationManager 根据当前的认证类型进行认证，认证时会根据提交的用户信息最终返回一个 SpringSecurity 的 UserDetails 对象，如果返回的 UserDetails 对象为空，则说明认证失败，抛出异常。

如果返回的 UserDetails 对象不为空，则返回 UserDetails 对象，最后 Authentication Manager 认证管理器返回一个被填充满了信息的 Authentication 实例，包括权限信息、身份信息、细节信息，但密码通常会被移除。

SecurityContextHolder 安全上下文容器存放填充了信息的 Authentication，认证成功后通过 SecurityContextHolder.getContext().setAuthentication()方法将 Authentication 设置到其中。

上述认证流程中，AuthenticationManager 接口是认证相关的核心接口，也是发起认证的出发点，它的实现类为 ProviderManager。而 Spring Security 支持多种认证方式，因此 ProviderManager 维护着一个 List<AuthenticationProvider> 列表，该列表中存放着多种认证方式，最终实际的认证工作是由 AuthenticationProvider 完成的。Web 表单的认证对应着 Authentication Provider，它的实现类为 DaoAuthenticationProvider，DaoAuthenticationProvider 内部又维护着一个 UserDetailsService 对象，该对象负责 UserDetails 的获取。Spring Security 提供的 InMemoryUserDetailsManager（内存身份、认证）JdbcUserDetailsManager（JDBC 身份认证）就是 UserDetailsService 的实现类，这两个实现类的主要区别在于是从内存还是从数据库加载用户信息。

7.3.2 Spring Security 自定义身份认证

尽管项目启动时，Spring Security 会提供默认的用户信息，可以快速认证和启动，但大多数应用程序都希望使用自定义的用户认证。Spring Security 进行认证时，DaoAuthentication Provider 会去对比 UserDetailsService 提取的用户密码与用户提交的密码，将匹配结果作为认证成功的关键依据，因此可以通过 Spring Security 内置的 UserDetails Service 实现类或者自定义的 UserDetailsService 进行自定义身份验证。针对自定义用户认证，Spring Security 提供了多种认证方式，常用的有内存身份认证、JDBC 身份认证和自定义 UserDetailsService 身份认证。下面对 Spring Security 的这 3 种自定义身份认证进行详细讲解。

1. 内存身份认证

Spring Security 的内存身份认证是最简单的身份认证方式，主要用于 Spring Security 安全认证体验和测试。以内存身份认证时，需要在 Spring Security 的相关组件中指定当前认证方式为内存身份认证。不同版本的 Spring Security 组件配置方式也不太相同。Spring Security 5.7.1 之前的版本中配置 Spring Security 的组件信息继承自 WebSecurityConfigurer Adapter 类并重写对应的方法，只需在重写的方法中设置相关组件信息即可。从 Spring Security 5.7.1 版本开始，Spring Security 将 WebSecurityConfigurerAdapter 类标注为过时，推荐直接声明配置类，并在配置类中直接定义组件的信息。

本书使用 Spring Boot 2.7.6，其对应的 Spring Security 版本为 5.7.5。自定义内存身份认证时，可以通过 InMemoryUserDetailsManager 类实现，InMemoryUserDetailsManager 是 UserDetailsService 的一个实现类，以便于在内存中创建一个用户。对此，只需在自定义配置类中创建 InMemoryUserDetailsManager 实例，在该实例中指定该实例的认证信息，并存入 Spring 容器即可，具体如下。

（1）创建配置类

在 chapter07 项目中创建名为 com.itheima.config 的包，并在该包下创建一个配置类 WebSecurityConfig，在该类中创建 UserDetailsService 类型的 InMemoryUserDetailsManager 实例对象交由 Spring 容器管理，具体如文件 7-6 所示。

文件 7-6 WebSecurityConfig.java

```
1 import org.springframework.context.annotation.Bean;
2 import org.springframework.context.annotation.Configuration;
3 import org.springframework.security.core.userdetails.User;
4 import org.springframework.security.core.userdetails.UserDetailsService;
```

```
5 import org.springframework.security.provisioning.InMemoryUserDetailsManager;
6 @Configuration
7 public class WebSecurityConfig  {
8     @Bean
9     public UserDetailsService userDetailsService() {
10        InMemoryUserDetailsManager users = new InMemoryUserDetailsManager();
11        users.createUser(User.withUsername("zhangsan")
12              .password("{noop}1234")
13              .roles("ADMIN")
14              .build());
15        return users;
16    }
17 }
```

在上述代码中，第 8～16 行代码通过 userDetailsService()方法向 Spring 容器中注册一个 UserDetailsService 实例对象，该实例对象的类型实际为 UserDetailsService 的子类 InMemoryUserDetailsManager。InMemoryUserDetailsManager 可以根据指定的用户信息在内存中创建用户，第 11 行代码指定用户名为 zhangsan；第 12 行代码指定用户密码为 1234，其中{noop}表示使用明文密码；第 13 行代码指定用户的角色为 ADMIN。

进行自定义用户认证时，需要注意以下几个问题。

- Spring Security 提供 BCryptPasswordEncoder、Pbkdf2PasswordEncoder、SCryptPasswordEncoder、DelegatingPasswordEncoder 等常用的密码编码器作为内置密码编译器，提交认证时会对输入的密码使用密码编译器进行加密，并与正确的密码进行校验。从 Spring Security 5 开始，默认的密码编译器为 DelegatingPasswordEncoder。如果不想对输入的密码进行加密，需要在密码前使用{noop}进行标注。
- 从 Spring Security 5 开始，自定义用户认证如果没有设置密码编码器，也没有在密码前使用{noop}进行标注，会认证失败，控制台会出现“IllegalArgumentException: There is no PasswordEncoder mapped for the id "null"”异常错误。
- 自定义用户认证时，可以定义用户角色 roles，也可以定义用户权限 authorities，在进行赋值时，权限通常是在角色值的基础上添加“ROLE_”前缀。例如，authorities("ROLE_ADMIN")和 roles("ADMIN")是等效的。
- 自定义用户认证时，可以为某个用户一次指定多个角色或权限，例如 roles("ADMIN", "COMMON")或者 authorities("ROLE_ADMIN"," ROLE_COMMON")。

Spring Security 是一个重要的安全框架，在开发过程中，需要从思想意识的高度对用户信息的安全给予重视，注重应用程序的安全性、可靠性和可维护性。特别是对于用户信息的管理和存储，需要使用合适的加密算法，保证用户信息不被泄露。

（2）验证内存身份认证

启动项目后，查看控制台输出的信息，发现没有默认安全管理时随机生成了密码。在浏览器中访问项目首页“ http://localhost:8080/ ”，效果如图 7-8 所示。

从图 7-8 可以看出，与之前一样，没有认证之前访问项目的资源会跳转到 Spring Security 提供的默认登录页面。

在图 7-8 中使用不是文件 7-6 中设置的用户信息进行登录，例如使用用户名：lisi，密码：1234，效果如图 7-9 所示。

图7-8 访问项目首页效果

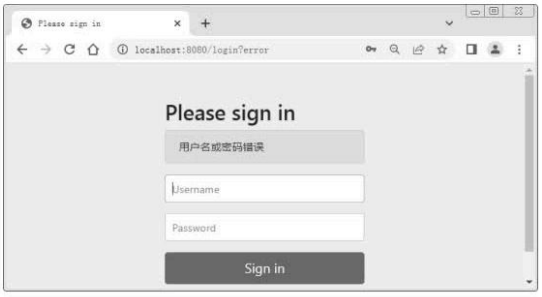

图7-9 使用错误用户信息登录（1）

从图 7-9 可以看出，使用错误的用户信息登录时，认证失败并在页面中提示“用户名或密码错误”。

在图 7-9 中使用文件 7-6 中设置的用户信息进行登录，效果如图 7-10 所示。

图7-10 项目登录成功效果（1）

从图 7-10 可以看出，使用文件 7-6 中设置的用户信息进行登录后认证成功，并跳转到原来想要访问的系统后台首页，说明通过内存身份认证方式实现了自定义身份认证。

实际开发中，用户都是通过注册和登录页面进行认证管理的，而非在程序内部使用内存管理的方式手动控制注册用户，所以上述使用内存身份认证的方式无法用于实际生产，只可供初学者测试。

2. JDBC 身份认证

JDBC 身份认证是通过 JDBC 连接数据库，基于数据库中已有的用户信息进行身份认证，这样避免了内存身份认证的弊端，可以实现对系统已注册的用户进行身份认证。JdbcUserDetailsManager 是 Spring Security 内置的 UserDetailsService 的实现类，使用 JdbcUserDetailsManager 可以通过 JDBC 将数据库和 Spring Security 连接起来。下面将对 JDBC 身份认证方式进行讲解。

（1）数据准备

JDBC 身份认证的本质是使用数据库中已有的用户信息在项目中实现用户认证服务，所以需要提前准备好相关数据。这里使用之前创建的名为 springbootdata 的数据库，在该数据库中创建三个表 user、priv 和 user_priv，并预先插入几条测试数据。准备数据的 SQL 代码如文件 7-7 所示。

文件 7-7 security.sql

```
# 选择使用数据库
USE springbootdata;
# 创建表 user 并插入相关数据
DROP TABLE IF EXISTS `user`;
```

```
CREATE TABLE `user` (
  `id` int(20) NOT NULL AUTO_INCREMENT,
  `username` varchar(200) DEFAULT NULL,
  `password` varchar(200) DEFAULT NULL,
  `valid` tinyint(1) NOT NULL DEFAULT 1,
  PRIMARY KEY (`id`)
) ENGINE=InnoDB AUTO_INCREMENT=4 DEFAULT CHARSET=utf8;
INSERT INTO `user` VALUES ('1', 'zhangsan',
       '$2a$10$7fWqX7Y010pMnyym/AHZX.3chIbnPZbj3N/iqcG4APCF.hC6CMh5a', '1');
INSERT INTO `user` VALUES ('2', 'lisi',
       '$2a$10$7fWqX7Y010pMnyym/AHZX.3chIbnPZbj3N/iqcG4APCF.hC6CMh5a', '1');
# 创建表 priv 并插入相关数据
DROP TABLE IF EXISTS `priv`;
CREATE TABLE `priv` (
  `id` int(20) NOT NULL AUTO_INCREMENT,
  `authority` varchar(20) DEFAULT NULL,
  PRIMARY KEY (`id`)
) ENGINE=InnoDB AUTO_INCREMENT=3 DEFAULT CHARSET=utf8;
INSERT INTO `priv` VALUES ('1', 'ROLE_COMMON');
INSERT INTO `priv` VALUES ('2', 'ROLE_ADMIN');
# 创建表 user_priv 并插入相关数据
DROP TABLE IF EXISTS `user_priv`;
CREATE TABLE `user_priv` (
  `id` int(20) NOT NULL AUTO_INCREMENT,
  `user_id` int(20) DEFAULT NULL,
  `priv_id` int(20) DEFAULT NULL,
  PRIMARY KEY (`id`)
) ENGINE=InnoDB AUTO_INCREMENT=5 DEFAULT CHARSET=utf8;
INSERT INTO `user_priv` VALUES ('1', '1', '1');
INSERT INTO `user_priv` VALUES ('2', '2', '2');
```

上述代码中，创建了三个表 user、priv 和 user_priv。其中，user 对应于用户表，priv 对应于用户权限表，user_priv 属于用户权限关联表。另外，使用 JDBC 身份认证方式创建用户/权限表和初始化数据时应，特别注意以下几点。

- 为了演示方便，在此只创建了用户、权限和用户权限关联表，并没有对用户角色和权限组进行扩展，此处的一个用户对应一种权限，同时也代表一种角色；
- 创建用户表 user 时，用户名 username 必须唯一，因为 Security 进行用户查询时是先通过 username 定位是否存在唯一用户的；valid 字段存储 tinyint 类型的值，用于校验用户身份是否合法（默认都是合法的）。
- 本案例使用 BcryptPasswordEncoder 作为密码编译器，文件 7-7 中初始化用户表 user 数据时，插入的用户密码 password 是对用户密码使用密码编译器编码后的密码，其中 $2a$10$7fWqX7Y010pMnyym/AHZX.3chIbnPZbj3N/iqcG4APCF.hC6CMh5a 是使用 BCrypt Password Encoder 密码编译器编译后的形式，对应的原始密码为 123456，因此，在自定义配置类中进行用户密码查询时，必须用与数据库密码统一的密码编码器进行编码。
- 初始化权限表 priv 数据时，权限 authority 值必须带有“ROLE_”前缀，而默认的用户角色值则是对应权限值去掉“ROLE_”前缀。

（2）配置依赖

打开 chapter07 项目的 pom.xml 文件，在该文件中添加 JDBC 的启动器依赖，具体如下。

```
<!-- JDBC 数据库连接启动器 -->
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-jdbc</artifactId>
</dependency>
<dependency>
   <groupId>com.mysql</groupId>
   <artifactId>mysql-connector-j</artifactId>
</dependency>
```

（3）设置配置信息

在项目 chapter07 中创建配置文件 application.yml，在该文件中设置数据库连接的相关配置信息，具体如文件 7-8 所示。

文件 7-8 application.yml

```
spring:
  datasource:
    url: "jdbc:mysql://localhost:3306/springbootdata?\
    characterEncoding=utf-8&serverTimezone=Asia/Shanghai"
    username: root
    password: root
```

（4）修改配置类

修改文件 7-6 中 WebSecurityConfig 配置类 userDetailsService()方法，将该方法创建的实例对象修改为 JdbcUserDetailsManager，修改后的代码如文件 7-9 所示。

文件 7-9 WebSecurityConfig.java

```
 1 import org.springframework.beans.factory.annotation.Autowired;
 2 import org.springframework.context.annotation.Bean;
 3 import org.springframework.context.annotation.Configuration;
 4 import org.springframework.security.core.userdetails.UserDetailsService;
 5 import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
 6 import org.springframework.security.crypto.password.PasswordEncoder;
 7 import org.springframework.security.provisioning.JdbcUserDetailsManager;
 8 import javax.sql.DataSource;
 9 @Configuration
10 public class WebSecurityConfig  {
11    @Autowired
12    private DataSource dataSource;
13    @Bean
14    public PasswordEncoder passwordEncoder() {
15       return new BCryptPasswordEncoder();
16    }
17    @Bean
18    public UserDetailsService userDetailsService() {
19       String userSQL ="SELECT username,password, valid " +
20             "FROM user WHERE username = ?";
21       String authoritySQL="SELECT u.username,p.authority " +
22             "FROM user u,priv p,user_priv up " +
23             "WHERE up.user_id=u.id AND up.priv_id=p.id and u.username =?";
```

```
24        JdbcUserDetailsManager users = new JdbcUserDetailsManager();
25        users.setDataSource(dataSource);
26        users.setUsersByUsernameQuery(userSQL);
27        users.setAuthoritiesByUsernameQuery(authoritySQL);
28        return users;
29    }
30 }
```

在上述代码中，第 11～12 行代码注入数据源对象 dataSource；第 13～16 行代码创建 BCrypt PasswordEncoder 密码编译器对象，并交由 Spring 管理，Spring 容器注入该对象后，会使用 BCryptPasswordEncoder 对象作为当前程序的密码编译器；第 17～29 行代码创建 JdbcUserDetailsManager 对象交由 Spring 管理，其中第 25 行代码用于设置数据源，第 26 行代码用于设置用户查询语句，第 27 行代码用于设置用户权限语句。

使用 JDBC 身份认证的方式进行身份认证时，需要让密码编译器与数据库中用户密码加密方式保持一致；同时需要加载 JDBC 进行认证连接的数据源 DataSource；最后，执行 SQL 语句，实现通过用户名 username 查询用户信息和用户权限。

需要注意的是，定义用户查询的 SQL 语句时，必须返回用户名 username、密码 password、是否为有效用户 valid 这三个字段信息；定义权限查询的 SQL 语句时，必须返回用户名 username、权限 authority 这两个字段信息。否则，进行认证时会出现 SQL 异常错误信息。

（5）效果测试

重启 chapter07 项目进行效果测试，项目启动成功后，通过浏览器访问“ http://localhost:8080/ ”后使用错误的数据库用户信息进行登录，例如使用用户名：zhangsan，密码：1234，效果如图 7-11 所示。

从图 7-11 可以看出，使用错误的用户信息登录时，认证失败并在页面中提示“ 用户名或密码错误 ”。

在图 7-11 中使用正确的数据库用户信息进行登录，效果如图 7-12 所示。

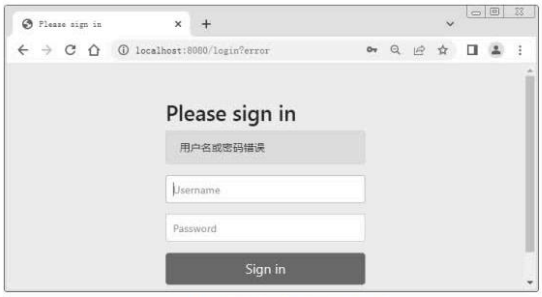

图7-11　使用错误用户信息登录（2）

图7-12　项目登录成功效果（2）

从图 7-12 可以得出，使用数据库中对应的用户信息进行登录后认证成功，说明通过 JDBC 身份认证方式实现了自定义身份认证。

3. 自定义 UserDetailsService 身份认证

使用 InMemoryUserDetailsManager 和 JdbcUserDetailsManager 进行身份认证时，其真正的认证逻辑都在 UserDetailsService 接口重写的 loadUserByUsername()方法中。对于一个完善的项目来说，通常会实现用户信息查询服务，对此可以自定义一个 UserDetailsService 实现类，重写该接口的 loadUserByUsername()方法，在该方法中查询用户信息，将查询到

的用户信息填充到 UserDetails 对象之后，返回该 UserDetails 对象，以实现用户的身份认证。下面通过案例对自定义 UserDetailsService 进行身份验证的实现进行演示，具体如下。

（1）创建实体类

在项目 chapter07 的 java 目录下创建包 com.itheima.chapter07.entity，在该包下创建用户实体类 UserDto 和权限实体类 Privilege，具体如文件 7-10 和文件 7-11 所示。

文件 7-10 UserDto.java

```
public class UserDto {
    private Integer id;       //用户编号
    private String username;      //用户名称
    private String password;     //密码
    private Integer valid;      //是否合法
     //……setter/getter 方法
}
```

文件 7-11 Privilege.java

```
public class Privilege {
    private Integer id;            //编号
    private String authority;      //权限
     //……setter/getter 方法
}
```

（2）创建用户持久层接口

在项目 chapter07 的 java 目录下创建包 com.itheima.chapter07.dao，在该包下创建用户持久层接口，在接口中定义查询用户和角色信息的方法，具体如文件 7-12 所示。

文件 7-12 UserDao.java

```
import com.itheima.chapter07.entity.Privilege;
import com.itheima.chapter07.entity.UserDto;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jdbc.core.BeanPropertyRowMapper;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Repository;
import java.util.ArrayList;
import java.util.List;
@Repository
public class UserDao {
    @Autowired
    JdbcTemplate jdbcTemplate;
    //根据用户名称查询用户信息
    public UserDto getUserByUsername(String username){
        String sql = "SELECT * FROM user WHERE username = ?";
        //连接数据库查询用户
        List<UserDto> list = jdbcTemplate.query(sql,
            new BeanPropertyRowMapper<>(UserDto.class),username);
        if(list !=null && list.size()==1){
            return list.get(0);
        }
        return null;
    }
    //根据用户 id 查询用户权限
    public List<String> findPrivilegesByUserId(Integer userId){
```

```
26        String sql = "SELECT u.username,p.authority " +
27              "FROM user u,priv p,user_priv up " +
28              "WHERE up.user_id=u.id AND up.priv_id=p.id and u.id =?";
29        List<Privilege> list = jdbcTemplate.query(sql,
30            new BeanPropertyRowMapper<>(Privilege.class), userId);
31        List<String> privileges = new ArrayList<>();
32        list.forEach(p -> privileges.add(p.getAuthority()));
33        return privileges;
34    }
35 }
```

在上述代码中，第 11～12 行代码注入 JdbcTemplate 对象；第 14～23 行代码定义的 getUserBy Username()方法用于根据用户名称查询用户信息；第 25～34 行代码定义的 findPrivilegesBy UserId()方法用于根据用户 id 查询用户权限。

（3）封装用户认证信息

在项目 chapter07 的 java 目录下创建包 com.itheima.chapter07.service，在该包下创建 User DetailsServiceImpl 类，该类实现了 UserDetailsService 接口，并在重写的 loadUserByUsername() 方法中封装了用户认证信息，具体如文件 7-13 所示。

文件 7-13　UserDetailsServiceImpl.java

```
1  import com.itheima.chapter07.dao.UserDao;
2  import com.itheima.chapter07.entity.UserDto;
3  import org.springframework.beans.factory.annotation.Autowired;
4  import org.springframework.security.core.userdetails.User;
5  import org.springframework.security.core.userdetails.UserDetails;
6  import org.springframework.security.core.userdetails.UserDetailsService;
7  import
org.springframework.security.core.userdetails.UsernameNotFoundException;
8  import org.springframework.stereotype.Service;
9  import java.util.List;
10 @Service
11 public class UserDetailsServiceImpl implements UserDetailsService {
12     @Autowired
13     UserDao userDao;
14     //根据用户名称查询用户信息
15     @Override
16     public UserDetails loadUserByUsername(String username) throws
17             UsernameNotFoundException {
18         //连接数据库根据用户名查询用户信息
19         UserDto userDto = userDao.getUserByUsername(username);
20         if(userDto == null){
21             //如果用户查不到，返回 null，会抛出异常
22             return null;
23         }
24         //根据用户的 id 查询用户的权限
25         List<String> privileges = userDao.findPrivilegesByUserId(userDto.getId());
26         //将 privileges 转换成数组
27         String[] privilegeArray = new String[privileges.size()];
28         privileges.toArray(privilegeArray );
29         UserDetails userDetails = User.withUsername(userDto.getUsername())
```